



University of Salerno

Department of Information and Electrical Engineering and
Applied Mathematics

Master's Degree in Computer Engineering

TechRate: Digital Platform for the Evaluation of PC Components

Group 01

Professor:

Carlo Mazzocca

Students:

Lucia Nancy Lavista - 0622702592

Alessandro Maruotto - 0622702601

Michele Penna - 0622702585

Academic Year 2025/2026

Contents

1	System and threat model	3
1.1	Context and motivation	3
1.1.1	The central role of reputation systems	3
1.1.2	The criticalities of current centralized systems	3
1.1.3	The need for a new paradigm: towards decentralization	4
1.1.4	Application Domain	4
1.2	Functional requirements	4
1.3	System entities	5
1.4	System proposal	6
1.5	Threat model	7
1.6	Desired security properties	8
2	System's Architecture	10
2.1	Overview and technology stack	10
2.1.1	Blockchain type	10
2.2	Identity management and onboarding	11
2.2.1	Seller Onboarding	11
2.2.2	User registration	12
2.3	Product Selection and Checkout	13
2.3.1	Review Uploading	16
2.4	Review Lifecycle Management	18
2.5	Utility Voting and Proof-of-Curation	20
3	System Verification	25
3.1	Methodology	25
3.2	Property-by-property verification	25
3.3	Threat-by-threat analysis	29
3.4	Design trade-offs and optimality	33
3.5	Residual Limitations and Future Work	34
3.5.1	Results Recap	35
4	Implementation and Evaluation	37
4.1	Technology Stack	38
4.1.1	Blockchain Layer (Smart Contracts and EVM)	38
4.1.2	Backend Issuer (Web2 E-Commerce Simulation)	39
4.1.3	Frontend Decentralized Application	39
4.1.4	Web3 Integration and Decentralized Storage	39
4.2	Smart Contract Implementation	39
4.2.1	Core Contracts Architecture	40
4.2.2	The Review Struct and On-Chain State	41
4.2.3	On-Chain Signature Verification	42

4.2.4	Curation Finalization and Reward Distribution	43
4.2.5	Deployment Mechanics and Post-Deploy Wiring	44
4.3	The Issuer Backend: Bridging Web2 and Web3	45
4.3.1	Proof of Purchase Generation and Selective Disclosure	45
4.3.2	Cryptographic Compatibility: Raw ECDSA over secp256k1	45
4.4	Decentralized Application (Frontend Demo)	46
4.4.1	Wallet Connection and DID Auto-Registration	46
4.4.2	The Submission Pipeline	47
4.4.3	Event-Driven Responsiveness and Development Tooling	48
4.5	System Evaluation and Testing	48
4.5.1	Test Architecture and Isolation Strategy	48
4.5.2	Test Coverage Summary	49
4.5.3	Off-Chain Signature Generation in Tests	51
4.5.4	Curation Window Simulation (Time Travel)	51
4.5.5	Test Execution Results	52

Workpackage 1

System and threat model

1.1 Context and motivation

1.1.1 The central role of reputation systems

In the current landscape of the digital economy and e-commerce platforms, review and reputation systems play a fundamental role. They represent the primary tool for establishing trust between buyers and suppliers who do not know each other, reducing transaction-related risks and improving the overall quality of online marketplaces. However, for this ecosystem to thrive, it is imperative that the feedback system is perceived as robust, transparent, and impartial.

1.1.2 The criticalities of current centralized systems

Despite their importance, traditional reputation systems suffer from several structural limitations due to their centralized nature:

- **Vulnerability and manipulation:** reputation data is stored on centralized servers managed by a single entity. This model exposes the system to tampering risks, as a dishonest (or compromised) platform could arbitrarily alter or censor scores without users' consent or awareness.
- **Data isolation:** a user's or supplier's reputation is typically confined within a single isolated database. This lack of interoperability prevents trustworthy suppliers from leveraging their reputation across different platforms, thereby fragmenting market trust.
- **Privacy threats and risk of retaliation:** in current platforms, rating activities are often strongly linked to users' public identities.
- **Fraud and malicious attacks:** centralized systems struggle to dynamically mitigate manipulation attempts, such as *multiple rating attacks* (where malicious users submit multiple reviews for a single transaction) and *abnormal rating attacks* (where extreme and unjustified ratings are used to artificially influence reputation scores).

1.1.3 The need for a new paradigm: towards decentralization

These challenges highlight the need for reputation systems that are transparent, tamper-resistant, and interoperable. In this context, Blockchain technology and Smart Contracts emerge as a promising solution, providing a distributed and immutable ledger capable of removing trust barriers between different platforms and decentralizing the management of reputation.

However, the direct adoption of blockchain technologies introduces a significant trade-off between the transparency of public ledgers and the privacy requirements of the involved actors. This project is motivated by the need to overcome this limitation by designing a decentralized reputation architecture that guarantees the authenticity and immutability of reviews while simultaneously integrating advanced cryptographic mechanisms to preserve buyer anonymity, prevent systematic fraud, and restore user control over personal data.

1.1.4 Application Domain

The target application domain for this architecture is the market for PC hardware components, specifically high-performance computing parts such as graphics processing units (GPUs), central processing units (CPUs), and power supply units (PSUs). This sector is structurally characterized by a high degree of information asymmetry between vendors and consumers, coupled with a tangible risk of economic fraud. Common deceptive practices within this domain include the sale of counterfeit components, unauthorized refurbished products, or hardware previously subjected to intensive, continuous workloads (e.g., cryptocurrency mining or server farms) and subsequently repackaged and resold as new.

In such a technically complex market, where the actual wear and tear of electronic components is not easily verifiable by the average user upon delivery, consumers rely heavily on peer reviews and community feedback to assess both product performance and seller reliability. Consequently, the integrity of the review ecosystem becomes critically important for protecting buyers and supporting informed purchasing decisions.

However, the effectiveness of this feedback loop is currently undermined by systemic vulnerabilities. The proliferation of unverified reviews, coordinated manipulation campaigns, and the inability to definitively distinguish genuine customer experiences from fabricated content severely degrade market transparency. Furthermore, the isolation of reputational data within individual, centralized e-commerce platforms prevents the establishment of a cohesive, market-wide trust metric. This fragmentation leaves buyers exposed to predatory sellers who can easily migrate across different marketplaces once their reputation is compromised on a single platform.

Therefore, there is a critical need to establish an evaluation environment where the authenticity of feedback is strictly tied to real commercial interactions, and where the curation of information is driven by verifiable trustworthiness rather than easily manipulated metrics.

1.2 Functional requirements

At an abstract level, the proposed system must satisfy the following functional requirements, regardless of the specific underlying technological implementation:

- **Proof of Purchase:** the system must guarantee that only users who have actually completed a financial transaction for a PC component are allowed to submit a review. This requirement aims to prevent spam and ensure the authenticity of feedback.
- **Public Verifiability and Censorship Resistance:** the management of reputation data must be independent of the e-commerce platform facilitating the transaction. While the platform acts as the trusted issuer of the purchase credential, the submitted feedback must reside on an immutable, decentralized ledger. This architectural separation guarantees that legitimate reviews cannot be hidden, modified, or deleted by the commercial entity to artificially manipulate its public reputation.
- **Reputation evaluation and incentive mechanisms:** the system must compute a reputation score for sellers based on the aggregation of user feedback. Furthermore, it should provide mechanisms to evaluate the usefulness and credibility of reviews, rewarding honest behavior while discouraging opportunistic or malicious actions.

1.3 System entities

The proposed system involves several categories of participants, each characterized by distinct roles, objectives, and capabilities:

- **Users / Consumers (raters):** these represent the purchasers of PC components. Their primary objective is to maintain full sovereignty over their personal data and privacy while being able to submit authentic and pseudonymous reviews. Users are assumed to be rational and incentive-driven rather than inherently malicious.
- **Sellers (rates):** commercial entities that sell hardware products. Their main interest is to receive ratings and reviews in order to build and maintain a trustworthy public reputation. Seller identities are public and verified through off-chain mechanisms.
- **E-Commerce platform:** the centralized marketplace where users purchase hardware components. It is assumed to be trusted. It is responsible for certifying that a user has successfully completed a financial transaction and subsequently issuing a verifiable Proof of Purchase via a cryptographically signed credential. It does not host the review database, thus losing the ability to censor or manipulate user feedback.
- **Certification Authorities (CAs):** trusted verification entities responsible for the onboarding of Sellers. The CA performs the real-world vetting of a seller's legal documentation (e.g., VAT number, Chamber of Commerce registration) outside the boundaries of the system. Upon successful verification, it issues cryptographic authorizations that logically bind the seller's verified legal identity to their digital identifier within the platform. This binding formally authorizes the seller to operate, register products, and accumulate reputation within the shared infrastructure.

1.4 System proposal

The proposed architecture is conceived as a decentralized reputation overlay designed to integrate with a centralized e-commerce marketplace. The foundational premise of this model rests on a clear separation of trust: the e-commerce platform is assumed to be a trusted entity strictly for the validation of commercial transactions. That is, the platform is trusted to truthfully certify that a specific user has completed the purchase of a specific item. However, the system deliberately removes the platform's exclusive control over the post-purchase evaluation phase, shifting the management of reviews and reputation to a transparent, shared infrastructure.

Crucially, to guarantee accountability and prevent manipulation, the infrastructure enforces a strict cryptographic entitlement policy. Rather than relying on traditional identity providers for consumers, the system utilizes an economic barrier to achieve Sybil resistance: every interaction must be backed by a mathematically verifiable proof of a financial transaction. Consumers operate pseudonymously, uniquely identified by their digital wallets and the purchase credentials they hold. Conversely, commercial entities (sellers) are formally vetted through recognized Certification Authorities. This hybrid approach ensures that feedback is inherently tied to real economic activity while preserving user privacy.

Building upon this separation of trust, the system performs three fundamental macro-functions:

- **Privacy-preserving purchase verification.** The system acts as a privacy-preserving bridge between verified purchases and the reputation layer. By leveraging the e-commerce platform's certification of the transaction, the architecture guarantees that the right to publish a review is strictly and exclusively bound to a verified purchase. This entitlement mechanism, combined with robust anti-replay safeguards on the decentralized ledger, eliminates the possibility of generating fabricated feedback. Furthermore, this validation is performed according to the principle of data minimization: consumers can mathematically prove their right to review without exposing their real-world identity, shipping information, or broader purchasing history to the public ecosystem.
- **Censorship resistance and immutable transparency.** The system ensures that reputational data are decoupled from the private infrastructure of the sales platform. As a consequence, the reputation of sellers or hardware components cannot be unilaterally manipulated by the marketplace or the vendor. Legitimate reviews cannot be hidden, modified, or removed to artificially improve public reputation or conceal defective products. Instead, all feedback becomes globally accessible, immutable, and publicly verifiable, substantially increasing market transparency while returning ownership of reputation data to the community.
- **Dual-metric reputation and community-driven trust.** The infrastructure establishes a resilient trust model through two independent reputation metrics. Rather than relying solely on the accumulation of product reviews, the system introduces a participatory curation process in which community members evaluate the usefulness of published reviews. This generates a second reputation metric associated with reviewers themselves. By weighting future evaluations according to each reviewer's historical trustworthiness, the protocol naturally promotes high-quality contributions while reducing the influence of malicious or manipulative

actors. Trust therefore emerges from mathematically aligned community incentives rather than from centralized moderation.

1.5 Threat model

The proposed system must be designed to operate securely in the presence of multiple adversarial actors, each with different motivations, risk tolerances, and attack strategies. Since this section defines the conceptual problem space prior to architectural implementation, adversaries are characterized by their behavioral patterns, incentives, and interaction with the platform's rules. The following sections describe the main threat categories considered in the system model.

Malicious user

This category represents actors who attempt to publish fraudulent or manipulative feedback without having legitimately acquired or experienced the evaluated item. Concretely, the adversary attempts to bypass the system's authentication and integrity rules in two ways:

- **Fake Buyers:** by generating numerous pseudonymous identities to simulate a crowd of fake buyers who never purchased the product.
- **Proof of Purchase Replay:** by reusing a single legitimate Proof of Purchase across multiple attempts to submit duplicate reviews for the exact same transaction.

Dishonest seller (bribery, self-dealing, and impersonation)

Commercial vendors inherently possess a strong economic incentive to maintain a high reputation standing and may engage in active manipulation to achieve this. Concretely, adversarial behavior at the vendor level manifests through three primary attack vectors:

- **Off-chain bribery:** secretly incentivizing consumers (e.g., via partial refunds, direct fiat payments, or discounts) to submit overly positive feedback or remove legitimate negative reviews, bypassing ledger integrity through external economic inducement.
- **Self-dealing:** purchasing their own inventory to generate valid, legitimate transaction receipts, subsequently utilizing them to publish synthetic positive reviews and artificially inflate their display ranking.
- **Identity hijacking and onboarding replay:** an external adversary or a rival actor attempting to intercept, replay, or forge authentication credentials during the preliminary merchant registration phase to illegitimately claim a commercial storefront or operate under a false corporate identity.

Curious platform (honest-but-curious issuer)

Within the system model, the e-commerce marketplace is assumed to be a trusted entity strictly regarding the validation of commercial transactions and the issuance of basic proofs of purchase. However, it represents a classical "honest-but-curious" (passive) adversary. While it faithfully follows protocol specifications without altering or forging

data, it leverages its central position in the transaction lifecycle to observe, log, and analyze traffic and user interactions.

Rational apathetic curator (curator apathy)

Apathy represents a passive game-theoretic threat where eligible users who possess legitimate transaction proofs and voting influence deliberately abstain from participating in the feedback evaluation and content curation process. In a decentralized governance or reputation model, widespread non-participation acts as an adversarial condition, as it drastically shrinks the active sample size required to establish a statistically meaningful and resilient consensus.

Reward farmer (incentive manipulation)

Farmers are economic opportunists who interact with the voting and curation layer purely as an extraction mechanism, completely disregarding the semantic quality or truthfulness of the feedback they evaluate. Instead of genuinely assessing whether a review is helpful or accurate, they employ programmatic strategies (such as random voting, automated copying of early voting trends, or blindly voting with the perceived majority) solely to unlock protocol rewards and incentives.

1.6 Desired security properties

The following security properties must be satisfied by any valid design of the proposed system. These properties are defined at an abstract level, independently of any specific implementation or technology choice. In WP 3, the proposed architecture will be evaluated against these requirements.

Integrity

No review, utility vote, or reputation score may be altered after submission and inclusion in the system without the modification being detectable by any participant. The system must provide *tamper-evidence*: any modification to a stored review or reputation metric must be deterministically observable by any verifying party. This property must hold even in the presence of a malicious storage node or a compromised e-commerce platform acting independently.

Authenticity and sybil resistance

Every operation performed within the system (e.g., review submission) must be attributable to an authorized participant. The architecture must prevent adversaries from manipulating reputation metrics by arbitrarily generating fictitious identities at scale (Sybil Attack). The system must enforce a mechanism that inherently bounds the ability to participate to verifiable, legitimate interactions within the ecosystem, ensuring that fake feedback cannot be mass-created without prohibitive constraints. At the same time, participant authenticity must be verifiable without permanently exposing real-world demographic identities on the public ledger.

Entitlement correctness (Proof of Purchase)

The system must enforce that a review for a specific hardware component can only be submitted by an entity possessing a valid and verifiable proof of a completed financial transaction for that exact item. It must be impossible for an adversary or a malicious seller to generate authentic-looking feedback without a legitimate underlying commercial exchange.

Double-submission prevention (non-replayability)

The system must guarantee that a single commercial transaction grants the right to submit exactly one review. Any attempt to reuse the same proof of purchase to submit multiple reviews or artificially inflate feedback volume must be deterministically detected and rejected.

Censorship resistance

No single entity, including the seller receiving the evaluation or the e-commerce platform facilitating the transaction, shall be able to prevent the publication of a legitimate and cryptographically valid review. Furthermore, no central authority shall be capable of arbitrarily hiding or deleting feedback from the immutable ledger to manipulate public perception.

Pseudonymity

The system must guarantee a strict decoupling between a consumer's real-world identity (or conventional Web2 account) and their public on-chain actions. When submitting feedback or participating in content curation, users must operate exclusively under cryptographic pseudonyms (such as Decentralized Identifiers or wallet addresses). The architecture must ensure that neither external observers, pattern-analysis attackers, nor the issuing e-commerce platform can link a publicly published review back to the consumer's legal identity, physical shipping address, financial details, or broader purchasing history.

Incentive alignment and metric integrity

The reputation scores of both sellers and users must be resilient to opportunistic manipulation. The system must ensure that the influence of a user's vote and the resulting visibility of a review are derived exclusively from a consensus-based assessment of quality.

The architecture should mathematically discourage behaviors such as farming and apathy, ensuring that reputation adjustments reflect long-term community consensus rather than mere interaction volume or activity frequency.

Workpackage 2

System's Architecture

2.1 Overview and technology stack

2.1.1 Blockchain type

In distributed ledger technologies, architectures are fundamentally categorized as either **permissioned** or **permissionless**.

A **permissioned blockchain** restricts network participation and consensus validation to a predefined set of authorized entities, often forming a consortium. While this allows for predictable throughput, it inherently reintroduces a degree of centralization and reliance on a trusted federation.

Conversely, a **permissionless blockchain** is entirely open and public; anyone can participate in the network, and the consensus is secured mathematically by the protocol without requiring any central administrative authority.

The proposed system adopts a **permissionless EVM-compatible blockchain network** (e.g., Ethereum). This architectural choice is strictly motivated by the need for absolute censorship resistance, true neutrality, and open interoperability.

If the infrastructure were permissioned and governed by a consortium of participating ECommerce platforms, the system could still be vulnerable to oligopolistic collusion (e.g., competitors agreeing to suppress systemic negative feedback) or gatekeeping (preventing new, rival marketplaces from joining the ecosystem). A fully public network ensures that the immutability of the state is guaranteed by a global, decentralized validator set, definitively stripping any commercial cartel of control over the feedback data and allowing any new platform to seamlessly integrate without requiring centralized approval.

To guarantee decentralization, cryptographic trust, and architectural scalability, the proposed system adopts a hybrid architecture that strictly separates identity management, computational validation, and heavy data storage across distinct decentralized layers:

- **Verifiable Data Registry (VDR):** the public blockchain acts as an immutable, globally accessible registry for Self-Sovereign Identity (SSI) management. Specifically, it serves as the foundational layer where participants' Decentralized Identifiers (DIDs) and their corresponding DID Documents are anchored. This allows any entity within the ecosystem to dynamically resolve public keys and verify cryptographic signatures (whether from users, vendors, or certification authorities) without

relying on centralized directories or third-party identity providers.

- **On-Chain Validation Mechanism:** rather than merely storing data, the blockchain functions as an autonomous state machine and cryptographic verifier. The smart contract layer natively enforces the protocol's governing rules, executing atomic checks to validate the authenticity of submitted Proof of Purchase (PoP) credentials and verifying participant entitlement. By processing digital signatures and tracking unique nullifiers directly on-chain, the system guarantees robust access control, double-submission prevention, and transparent consensus execution in a completely permissionless and trustless environment.

To ensure architectural efficiency and decouple computational logic from heavy data persistence, the system integrates the **InterPlanetary File System (IPFS)**, a peer-to-peer distributed storage network. All bulky payloads (such as review texts, titles, and multimedia evidence) are stored off-chain on IPFS. The underlying cryptographic Content Identifier (CID) generated by IPFS is then permanently anchored to the blockchain state.

This hybrid approach preserves the absolute immutability and verifiable integrity of the reviews while keeping the blockchain footprint minimal and lightweight.

2.2 Identity management and onboarding

2.2.1 Seller Onboarding

This flow is executed once per vendor wishing to join the ecosystem. It leverages Self-Sovereign Identity (SSI) principles to ensure verifiable identity binding without relying on centralized, non-transparent verification processes.

Preliminary Phase

To participate in the ecosystem, the vendor installs a **Digital Wallet**, which generates a cryptographic key pair ($P_{pubVENDOR}$, $P_{privVENDOR}$). Simultaneously, the wallet derives a **Decentralized Identifier (DID)** to serve as the vendor's unique, immutable identifier on the blockchain.

To make this identity resolvable by other participants, the wallet submits a transaction to the **DID Registry** smart contract, anchoring the vendor's public key ($P_{pubVENDOR}$) on-chain. This transaction is signed by the vendor's own private key. From this point on, any party can resolve the DID of the vendor to its corresponding DID Document by querying the registry, without relying on any centralized directory.

Verifiable Credential Request and Issuance

The vendor authenticates via SPID (or similar eIDAS-compliant systems) at a trusted **Certification Authority (CA)**, such as the Camera di Commercio. During this verification, the vendor submits his DID along with the necessary legal documentation.

Upon successful verification, the CA issues a Verifiable Credential **Vendor_VC** in JWT format, including:

- **vendorDID:** the Decentralized Identifier representing the vendor. This ensures standard-compliant entity referencing and allows downstream verifiers to resolve the associated cryptographic keys via the DID Document.

- **legalName**: The registered legal name or corporate name of the vendor's company (e.g., "TechCo S.r.l.").
- **vatNumber**: the Value-Added Tax (VAT) number, which serves as a unique business identification number for tax and regulatory purposes. This field formally proves the legitimate fiscal existence of the commercial entity within its jurisdiction, effectively mitigating the risk of fraudulent onboarding by non-vetted or shell companies.

The **Vendor_VC** Credential is then signed by the CA.

Registration on the ECommerce Service

The vendor submits a formal request to the Ecommerce Service, including the **Vendor_VC** Credential.

The ECommerce service acts as a Verifier, performing the following tasks:

- Using the DID Document registered on-chain, verifies the digital signature within the JWT to ensure it was issued by the trusted CA.
- Challenges the vendor to sign a **nonce** using $P_{priv_{VENDOR}}$. The service resolves the **vendorDID** on-chain to retrieve the vendor's public key and validates the signature, ensuring the submitter actually owns the credential and didn't simply steal the JWT file.
- Confirm that there are no other vendors already registered with the same **vatNumber**.

Upon successful validation, the vendor is finally registered on the ECommerce service and is authorized to manage their storefront within the ECommerce platform.

On-Chain Vendor Registration

Once the vendor has been successfully validated by the ECommerce Service, the platform records the vendor's existence within the protocol itself, on the blockchain.

To this end, the ECommerce Service submits a transaction to the smart contract, providing the relevant fields extracted from the **Vendor_VC**: the **vendorDID**, the **legalName**, and the **vatNumber**.

This transaction is signed and submitted by the ECommerce Service, not by the vendor: since the platform is the party that has just performed the verification, it is also the party responsible for attesting the outcome on-chain.

2.2.2 User registration

At this initial stage, in order to make a purchase, the user must register on the ECommerce platform.

It is important to note that the system does not enforce a strict one-to-one relationship between a user's identity and their account; in fact, users can choose to create separate profiles for each wallet they own.

The following paragraphs detail the steps required to complete this process.

Wallet Initialization and DID Derivation

The user installs a digital wallet client on their personal device, which generates a secure asymmetric cryptographic key pair $(P_{priv_{USER}}, P_{pub_{USER}})$.

The user's public wallet address, denoted as W_{user} , is derived from the corresponding public key $P_{pub_{USER}}$.

To ensure full compliance with the W3C Verifiable Credentials data model, this address is represented as a Decentralized Identifier (DID), which serves as the user's formal pseudonymous identifier within the ecosystem.

As with vendor onboarding, the wallet submits a transaction to the on-chain **DID Registry**, anchoring the user's public key $(P_{pub_{USER}})$ so that the corresponding DID Document becomes publicly resolvable.

The private key remains exclusively within the user's local secure environment and is never disclosed to external entities. Consequently, no third party can impersonate the user or generate valid cryptographic signatures on the user's behalf.

Web2 Registration and Identity Binding

The user registers on the ECommerce platform using conventional Web2 authentication mechanisms, such as an email–password combination or an OAuth-based login.

During the account registration process, the user is requested to provide their Decentralized Identifier (DID), or alternatively the underlying wallet address, which the platform deterministically resolves into the corresponding DID representation.

To prevent impersonation and cryptographically prove ownership, the platform issues a unique challenge (a random **nonce**) that the user must sign using the private key $P_{priv_{USER}}$ associated with their wallet.

Upon receiving the signed nonce, the backend performs a standard cryptographic verification to ensure that the signature is valid and corresponds to the public key associated with the initially declared DID.

If the signature is successfully validated, the proof of ownership is confirmed, and the platform securely binds the Web2 account to the Web3 DID in its database. An invalid signature, however, results in an immediate rejection.

The ECommerce platform acts as the trusted authority responsible for establishing this identity binding. Specifically, it associates the provided DID with the user's internal account profile within its proprietary off-chain database.

This off-chain linkage constitutes a fundamental prerequisite for the remainder of the protocol, as it enables the platform to unambiguously determine which blockchain identity is entitled to receive a Proof of Purchase credential whenever a purchase is successfully completed.

2.3 Product Selection and Checkout

The user accesses the ECommerce platform through their conventional Web2 account (e.g., email/password or OAuth authentication) and purchases a hardware component, for example `productID = "GPU-RTX4090-ASUS-ROG"`.

Order Fulfillment and Delivery

The ECommerce platform processes the payment and manages the order fulfillment process. To ensure that only users who have physically received and used the purchased hardware are allowed to contribute to the reputation system, the issuance of the credential is intentionally postponed until the delivery has been successfully confirmed by the courier.

Proof of Purchase VC Construction (SD-JWT)

Once delivery is confirmed, the platform generates a **Proof of Purchase (PoP)** as a W3C Verifiable Credential encoded using the SD-JWT format.

To comply with both standard W3C specifications and the principle of selective disclosure, the credential is composed of different logical blocks of claims. The ECommerce platform includes all real order details (acting as a valid commercial receipt), but thanks to the SD-JWT architecture, the user will be able to cryptographically hide sensitive data when presenting the proof to the smart contract.

The credential comprises the following fields:

1. Standard Metadata These mandatory fields define the technical validity of the credential:

- **iss (Issuer)**: the DID of the ECommerce platform issuing the receipt.
- **sub (Subject)**: the user's Decentralized Identifier (**User DID**). It is required to cryptographically bind the review submission to the credential owner.
- **iat (Issued At)**: the timestamp indicating when the delivery was confirmed and the credential was generated.
- **jti (JWT ID)**: a unique identifier for the credential itself

2. Disclosed Claims (Public Essential Fields)

- **productID**: the universal identifier (e.g., GTIN/EAN or MPN) of the purchased item. This field is publicly exposed because it is essential for the smart contract to categorize the review and update the product's average rating without revealing the user's broader order history.
- **vendorDID**: the Decentralized Identifier of the vendor who sold the purchased product. Disclosing this identifier allows the smart contract to unequivocally bind the review to the correct seller for reputation tracking ($\mathcal{R}_{\text{vendor}}$) and reply authorization.
- **reviewNullifier**: a unique hash value that prevents double-spending of the review right. It is computed off-chain by the issuer as:

$$\text{Nullifier} = \text{Hash}(\text{User_DID} + \text{ProductID} + \text{Secret_Salt})$$

This field is publicly exposed on the blockchain, allowing the smart contract to verify that a review has not been submitted previously for the same product-user pair, while keeping the underlying **User_ID** and **Secret_Salt** confidential.

3. Hidden Claims (Private Sensitive Fields)

- **orderID**: the internal reference number of the order.
- **shippingAddress**: the physical address where the hardware component was delivered.
- **purchasePrice**: the fiat amount paid (e.g., "1850.00").
- **currency**: the currency used for the transaction (e.g., "EUR").
- **paymentMethod**: the payment method utilized (e.g., "Credit Card" or "PayPal").

After constructing the credential with these fields, the ECommerce platform signs the complete SD-JWT using a digital signature algorithm.

Note

Architectural Rationale: Why SD-JWT? It is worth highlighting the architectural motivation behind adopting the W3C SD-JWT (Selective Disclosure JWT) format rather than a conventional standard JWT.

If the sole purpose of the Proof of Purchase credential were merely to authorize the review submission within this specific smart contract, a lightweight standard JWT containing strictly the essential disclosed claims (**productID**, **vendorDID**, and **reviewNullifier**) would have been technically sufficient.

However, within our domain, the Proof of Purchase is conceived as a multi-purpose, self-sovereign digital document that functions as a formal commercial receipt and warranty token. The consumer may need to utilize this complete credential for several off-chain operations—such as claiming hardware warranties with the manufacturer, verifying ownership for technical support, tax accounting, or proving authenticity during second-hand resale. These secondary use cases inherently require the inclusion of comprehensive transaction details, such as the physical shipping address, delivery date, and fiat purchase price.

By encoding the credential as an SD-JWT, the ECommerce platform resolves this trade-off: it issues a single, complete commercial receipt that maintains its utility across the broader digital economy, while empowering the consumer to exercise *selective disclosure*. When presenting the proof to the DApp for review submission, the user cryptographically conceals sensitive personal attributes (e.g., their home address and financial details) from the public ledger, exposing strictly the minimal parameters required by the protocol without sacrificing the credential's broader utility.

Credential Delivery and Protocol Deferral

The signed SD-JWT, together with any selective disclosure information required for future claim revelation, is securely transmitted to the user's digital wallet through an off-chain communication channel. The credential remains exclusively under the user's control until it is voluntarily presented to the DApp.

At this stage, no blockchain transaction is performed, and no on-chain state is created. The user receives their valid Proof of Purchase but is not yet registered within the protocol's smart contract. The actual on-chain registration is deferred until the exact moment the user decides to publish a review, utilizing a *Lazy Registration* approach. The

technical implementation, flow, and benefits of this lazy registration mechanism will be thoroughly detailed in the following sections.

2.3.1 Review Uploading

Once the user possesses a valid **Proof of Purchase**, they are cryptographically authorized to submit a review for the corresponding product through the decentralized application (DApp).

The DApp acts as an intermediary between the user and IPFS. Through the DApp, the user submits the review title, the review text, a rating ranging from 1 to 5, and optionally attaches multimedia evidence, such as photographs or videos of the purchased hardware component.

IPFS Upload

The first phase of the review submission process consists of the upload to **InterPlanetary File System (IPFS)**.

The DApp structures the review into a JSON document before publishing it to IPFS. A representative example of the review is shown below.

```
{
  "productID": "GPU-RTX4090-ASUS-ROG",
  "vendorDID": "did:method:0x...",
  "authorAddress": "0xUserAddress...",
  "language": "en",
  "review": {
    "title": "Excellent GPU",
    "text": "Excellent GPU: silent cooling, temp. below 70°C under load.",
    "score": 5,
    "mediaHashes": [
      "QmImg1...",
      "QmVideo1..."
    ]
  },
  "submittedAt": "2026-07-01T10:30:00Z"
}
```

The document is uploaded to IPFS, which deterministically computes a **Content Identifier (CID)** corresponding to the cryptographic hash of the stored content.

To minimize blockchain storage costs, the review itself is never persisted on-chain. Instead, only the resulting CID is stored as a lightweight on-chain reference, as detailed in Section 2.3.1.

Review Submission

The user's wallet already holds the Proof of Purchase, issued by the ECommerce platform at the moment delivery was confirmed. This credential is encoded as an SD-JWT whose payload combines two kinds of claims: a small set of **plaintext claims** which remain directly readable in the signed JWT, and a set of **hidden claims**, represented only as digests in the `_sd` array.

These hidden claims are never disclosed as part of the review submission protocol: they remain available to the user for purposes entirely external to the protocol (e.g., returns or disputes handled directly with the platform), and their corresponding values and salts are simply never transmitted in what follows.

To submit a review, the user, through the DApp, first composes the review content (title, text, score, optional media). The DApp uploads this content to IPFS, obtaining the corresponding **CID**.

The DApp then constructs the review submission transaction, attaching:

- the **Proof of Purchase**, exactly as received from the ECommerce platform;
- the numerical **score**;
- the IPFS **CID** obtained above.

The user signs this transaction with their own private key $P_{privUSER}$ and submits it directly to the smart contract.

Smart Contract Verification

Upon receiving the transaction, the smart contract executes a sequence of atomic verification checks. Failure of any single check causes the entire transaction to revert, leaving the on-chain state unchanged.

So, after verifying the digital signature of the user who has submitted the transaction, the Smart Contract performs the following checks:

Check 1: Issuer Signature Verification The contract verifies the signature over the attached Proof of Purchase against the public key of a registered, trusted ECommerce issuer P_{pubEC} . Only the plaintext claims (**userID**, **productID**, **vendorID**, **reviewNullifier**) are read at this stage; the **_sd** array is never expanded, since none of the hidden claims are relevant to review submission. If the signature does not verify against any trusted issuer, the transaction reverts.

Check 2: Sender-Subject Correspondence The contract verifies that the address which signed and submitted the transaction matches the **userID** claim of the verified SD-JWT. This ensures that only the legitimate holder of the Proof of Purchase, not anyone who might obtain a copy of it, can use it to submit a review, since the transaction itself is a form of authentication: only the owner of the corresponding private key could have produced a valid signature over it. If the sender does not match the credential's subject, the transaction reverts.

Check 3: Nullifier Verification The contract verifies that the **reviewNullifier** included in the transaction has not been previously recorded, by querying the on-chain nullifier registry. If the nullifier has already been spent, the transaction reverts, preventing the same Proof of Purchase from being used to submit more than one review. If the nullifier has not been seen before, it is atomically marked as spent as part of the same transaction.

Only once all checks succeed does the contract proceed to update its state and finalize the review submission, as detailed in the following sections.

Lazy Registration and Token Minting

Before storing the review, the contract checks whether the **reviewer** has previously submitted at least one review.

If this is the reviewer's first interaction with the protocol, a **Lazy Registration** procedure is automatically executed as part of the same transaction. The contract:

- creates an entry for the reviewer's address in the reputation registry, initializing their reputation score to $\mathcal{R}_0 = 1$;
- mints one "welcome" **Reputation Token** to the reviewer's address.

Both the reputation mechanism and the Reputation Token will be discussed in the following section.

On-Chain State Update and IPFS Persistence

If every verification succeeds, the transaction is finalized atomically. The smart contract:

- permanently records the **reviewNullifier**, preventing future reuse of the corresponding purchase credential;
- stores the review's minimal metadata (specifically the reviewer's address, the product identifier, the vendor identifier, the current IPFS CID, the numerical score, and the submission timestamp) enabling downstream operations such as review modification, revocation, and vendor reply authorization;
- records the association between the review's CID and its corresponding product and vendor identifiers, enabling efficient correlation between reviews, products, and vendors without duplicating full strings in storage.

2.4 Review Lifecycle Management

A review transitions through the following states:



Figure 2.1: Review lifecycle and state transitions.

Modification

Modifications are permitted within 3 hours of initial publication, up to a maximum of one update. As with submission, modification is performed **directly by the reviewer**: the user signs and submits the transaction themselves, and the smart contract verifies its authenticity natively, without relying on any external credential or intermediary. The modification flow proceeds as follows:

1. The user, through the DApp, composes the revised review text and any updated media.
2. The DApp uploads the revised content to IPFS, obtaining a new CID (**newCID**).

3. The user submits a transaction directly to the smart contract, containing the identifier of the review to be modified and the new IPFS CID.
4. The smart contract verifies that the transaction's sender matches the reviewer's address stored on-chain for that review, that fewer than 3 hours have elapsed since the review's original publication, and that no modification has been made to it yet.
5. The contract updates the active CID to **newCID**, while the previous CID remains permanently part of the on-chain transaction history. The vendor and product identifiers associated with the review are left unchanged, since a modification only revises the review's content, not its subject. The contract also resets the review's upvote and downvote counters, and reintroduces it into the curation queue as new content.

Historical transparency is preserved: the previous CID remains permanently recorded and publicly verifiable as part of the blockchain's transaction history, although the consortium actively pins only the most recent content version.

Revocation

The author may revoke a review at any time before the closure of its **30-day curation window** (Section 2.5), by signing and submitting a formal revocation transaction directly to the smart contract. Revocation is not permitted once this window has closed.

Since Reputation Tokens are minted to consensus-aligned voters only at closure, based on the votes cast on that specific review, allowing revocation afterwards would let a review be removed after it has already produced irreversible effects on other participants' token balances and reputation scores, with no way to consistently unwind them.

The ReputationToken mechanism will be discussed in the following sections.

The contract verifies that the transaction's sender matches the reviewer's address stored on-chain for that review and that the review's curation window is still open, then updates the review's on-chain record by marking it as revoked and triggers the immediate exclusion of the review from future seller reputation aggregations.

Since the review is removed before its curation window closes, no Reputation Tokens will ever be minted on the basis of votes cast on it, and any votes already recorded against it are simply discarded from the pending consensus computation. Regarding data availability, the original CID and all associated metadata remain permanently recorded and mathematically verifiable as part of the blockchain's transaction history.

Vendor Reply Mechanism

To foster a transparent and interactive ecosystem, the protocol allows commercial vendors to publicly reply to reviews associated with their products.

To prevent spam and maintain thread clarity, the system strictly enforces a "one reply per review" policy. Unlike the 30-day curation window enforced for utility voting, vendor replies are not restricted by any temporal deadline, allowing vendors to address feedback at any stage of the review's lifecycle.

Like modification and revocation, vendor replies are submitted **directly by the vendor**.

The mechanism proceeds as follows:

1. **Off-Chain Preparation:** the vendor drafts the response through the DApp frontend. The text is structured into a JSON document and uploaded to IPFS, which returns a unique Content Identifier (**replyCID**).
2. **On-Chain Submission:** the vendor signs and submits a transaction directly to the smart contract, providing the target review's identifier and the newly generated **replyCID**.
3. **Smart Contract Verification:** upon receiving the transaction, the contract executes a sequence of atomic checks. Failure of any check causes the transaction to revert:
 - The contract verifies that the transaction's sender corresponds to an active, registered vendor identity in the on-chain vendor registry.
 - Having established, in the previous check, the **vendorDID** associated with the sender's address in the vendor registry, the contract verifies that this **vendorDID** matches the one already recorded on-chain for the target review at submission time. This prevents a vendor from replying to reviews associated with a different seller.
 - The contract verifies that the target review has not been revoked by the reviewer.
 - The contract verifies that no reply has been recorded yet for the target review. If one already exists, the operation is aborted.
4. **State Update:** upon successful validation, the contract permanently records that the target review has received a reply, together with the corresponding **replyCID**, locking the reply capability for that review.

Interaction with Review Modifications In alignment with the protocol's design principles, the submission of a vendor reply does not freeze or terminate the reviewer's 3 hour modification window. If a user modifies their review after a vendor has already posted a reply, the modification is processed normally, updating the active CID. While DApp frontends will display the vendor's response alongside the newly updated review content, the historical immutability of the blockchain guarantees that any third party or arbitrator can independently verify that the vendor's response was originally submitted against the previous CID.

2.5 Utility Voting and Proof-of-Curation

Voting eligibility and submission

A voter wishing to cast a utility vote on a review uses the DApp frontend to attach their Proof of Purchase (SD-JWT) and specify their evaluation (upvote or downvote). The user then signs the transaction with their own private key and submits it directly to the smart contract.

Upon receiving the transaction, the smart contract executes a sequence of atomic on-chain verifications:

1. **Credential and Product Validation:** the contract verifies the cryptographic signature of the trusted ECommerce issuer on the attached Proof of Purchase. It then confirms that the **productID** within the credential strictly matches the **productID** of the review being evaluated. This guarantees that only verified owners of that specific hardware component can participate in the curation of its reviews, preventing Sybil attacks while preserving on-chain pseudonymity.
2. **Sender Authentication:** the contract verifies that the blockchain address submitting the transaction corresponds to the **userDID** claim inside the Proof of Purchase. This prevents third parties from intercepting or reusing someone else's credential to cast unauthorized votes.
3. **Per-Review Uniqueness:** to prevent double-voting without consuming the user's general review right (the purchase nullifier is deliberately not spent during utility voting), the contract checks an on-chain registry that permanently links the voter's address to the target review identifier. If the voter has already cast a vote on this specific review, the transaction is rejected; otherwise, the new vote is recorded atomically.

Once eligibility and uniqueness are verified, the system counts the vote according to the user's current effective weight. The effective weight W of a voter is mathematically defined as:

$$W = \log(1 + \mathcal{R}_{\text{voter}})$$

where $\mathcal{R}_{\text{voter}}$ represents the voter's current reputation score. A user with $\mathcal{R}_{\text{voter}} = 0$ has an effective weight of 0 and cannot influence the curation outcome until they establish a track record of consensus-aligned behaviour. Initial reputation upon registration is set to $\mathcal{R}_{\text{voter}} = 1$, granting a minimal but non-zero starting weight of $\log(2) \approx 0.693$.

The global review counters for upvotes (U) and downvotes (D) are consequently updated as follows:

$$\begin{cases} U \leftarrow U + W & \text{if the vote is "useful" (1)} \\ D \leftarrow D + W & \text{if the vote is "not useful" (0)} \end{cases}$$

The logarithmic function compresses the influence of high-reputation users, preventing dominance by a small subset of participants while still rewarding historically correct behavior. Following the state update, the system cryptographically logs the vote's metadata to ensure public auditability.

Curation window closure and reputation update

After the 30-day window elapses, any participant or automated keeper can trigger the finalization phase. The system verifies the elapsed time on-chain and proceeds to compute the weighted consensus score (C):

$$C = \frac{\sum_i (W_i \times V_i)}{\sum_i W_i}$$

where W_i is the effective weight of voter i and $V_i \in \{0, 1\}$ is their respective vote.

For each participating voter i , their reputation score ($\mathcal{R}_i$) is dynamically updated based

on their alignment with the global consensus:

$$\mathcal{R}_i \leftarrow \begin{cases} \mathcal{R}_i + \Delta^+ & \text{if } |V_i - C| < \theta \\ \mathcal{R}_i - \Delta^- & \text{otherwise} \end{cases}$$

The parameters Δ^+ , Δ^- , and the threshold θ are defined at contract deployment time. This mechanism incentivises users to vote genuinely: participants who vote at random or in bad faith tend to diverge from consensus and lose reputation, reducing their influence on future curation rounds. To mathematically discourage random voting and farming, the parameters must be calibrated off-chain prior to deployment to enforce risk asymmetry, ensuring that the reputational penalty (Δ^-) strictly outweighs the reward (Δ^+).

Protocol Parameters Definition and Calibration. To ensure the fairness and security of the curation mechanism, the protocol relies on three foundational parameters that must be calibrated prior to deployment:

- **Consensus Tolerance Threshold (θ):** This parameter ($\theta \in (0, 1)$) acts as the allowable margin of error for voters. Since individual votes are binary ($V_i \in \{0, 1\}$) and the final consensus score is a weighted average ($C \in [0, 1]$), θ defines how close a vote must be to the average to be considered correct. For example, if $\theta = 0.25$ and the consensus score is $C = 0.80$, any user who voted 1 is deemed aligned (since the difference is only 0.20, which is less than θ), while users who voted 0 are penalized.
- **Reputation Reward (Δ^+) and Penalty (Δ^-):** These parameters dictate the amount of reputation points gained or lost after a curation window closes. Δ^+ is the reward given to users who vote in alignment with the consensus, while Δ^- is the penalty applied to those who vote against it. To deter automated bots and random guessing, the protocol enforces an asymmetric rule where penalties are strictly larger than rewards ($\Delta^- > \Delta^+$). This ensures that careless or malicious voting results in a net loss of reputation over time.

Token minting upon finalization. At the closure of each curation window, the finalization process automatically triggers the issuance of Reputation Tokens for voters whose evaluation aligned with the final consensus. These tokens are implemented as Soulbound Tokens (SBTs), meaning all transfer operations are permanently disabled to prevent secondary markets for voting influence. Their exclusive purpose is to provide economic incentives without altering protocol mechanics.

The issuance logic defining the amount of tokens minted for voter i (T_i) is mathematically determined as:

$$T_i = \begin{cases} \lfloor \Delta^+ \times \mathcal{R}_i \times k \rfloor & \text{if } |V_i - C| < \theta \\ 0 & \text{otherwise} \end{cases}$$

where k is a scaling parameter set at contract deployment time. No tokens are minted for discordant voters, who incur only the reputational penalty Δ^- .

This design satisfies three explicit properties:

- **Proportionality:** the token amount is proportional to both Δ^+ and the voter's current reputation score ($\mathcal{R}_i$), rewarding users with a longer history of correct behaviour.

- **Asymmetry:** tokens are issued *only* to consensus-aligned voters, structurally reinforcing the incentive toward honest voting.
- **Deferred emission:** token generation occurs exclusively upon the closure of the curation window, preventing rewards based on an early majority that might be reversed later.

Algorithmic visibility

The visibility score of each review is computed as:

$$\text{visibility_score}(\text{review}) = \sum (\text{weight}_i \times \text{upvote}_i) - \sum (\text{weight}_i \times \text{downvote}_i) = U - D$$

This value is read directly from the smart contract state by the dApp frontend and used to sort reviews for display. No off-chain ranking logic is involved: ordering is **deterministic**, **publicly verifiable**, and **non-manipulable** by any platform operator.

Seller and product reputation aggregation

Upon the closure of each curation window, the system dynamically updates the aggregated reputation scores for both the commercial vendors and the specific hardware components. This aggregation is computed as a weighted average of the numerical scores from all valid, active, and non-revoked reviews. The reputation score for a given seller is mathematically formulated as follows:

$$\mathcal{R}_{\text{vendor}} = \frac{\sum_{i \in \mathcal{V}} (S_i \times W_i)}{\sum_{i \in \mathcal{V}} W_i}$$

Where:

- $\mathcal{V}$ represents the subset of all active, non-revoked reviews submitted for the target vendor. Crucially, to protect the ecosystem from spam and coordinated defamation (e.g., review bombing), the protocol enforces a strict quality filter: any review whose final algorithmic visibility score falls below zero is mathematically excluded from the set $\mathcal{V}$. Consequently, reviews deemed “not useful” by the community consensus lose their capacity to impact the seller’s aggregate reputation.
- S_i is the numerical rating score (1 to 5) assigned in review i .
- W_i corresponds to the weight of the review, defined as the reviewer’s historical **reputationScore** captured precisely at the time of submission ($\mathcal{R}_{\text{at_submission}}$).

An identical weighted paradigm, including the strict exclusion of negative-utility reviews, is applied symmetrically to compute individual product reputation scores ($\mathcal{R}_{\text{product}}$) based on their respective product identifiers. By design, reviews submitted by users who possessed a higher reputation at the moment of submission inherently carry significantly greater leverage in the final public metrics.

Burn-to-Redeem Mechanism

To redeem tokens, a participant submits a transaction specifying the destination ECommerce platform and the number of tokens to redeem. The protocol strictly enforces a **Minimum Redemption Threshold** (T_{min}) defined at the smart contract level. This threshold mathematically prevents the exploitation of onboarding incentives for immediate economic gain.

The protocol executes the following atomic sequence:

1. Verifies that the user's available token balance is greater than or equal to the minimum threshold (balance $\geq T_{\min}$).
2. Burns the requested amount of tokens (which does not modify the participant's reputation score $\mathcal{R}$).
3. Emits a publicly verifiable redemption event containing the user's wallet address, the destination platform, and the exact amount of burned tokens.

Off-Chain Discount Resolution

Upon detecting a valid redemption event on the blockchain, the ECommerce platform identifies the user by retrieving the identity link established during registration. To prevent replay attacks, the system verifies that the transaction has not been processed previously, and then automatically credits the corresponding discount value directly to the user's account.

Workpackage 3

System Verification

3.1 Methodology

The security analysis of the proposed architecture is conducted through a structured and comprehensive approach designed to verify the resilience of the system against the threats identified in the system model. The methodology proceeds along two complementary analytical directions, followed by a critical evaluation of the architectural choices:

- **Property-by-property verification:** for each security property defined in the system model, the analysis identifies and examines the specific cryptographic and architectural mechanisms introduced in the proposed solution (WP2) that guarantee its fulfillment.
- **Threat-by-threat walk-through:** for each adversary category outlined in the threat model, the evaluation traces the theoretical attack scenarios step-by-step within the implemented architecture. This process demonstrates at which specific stage the attack is mitigated or, conversely, explicitly acknowledges any residual risks that are structurally accepted within the trust model.
- **Design trade-offs and optimality:** the analysis concludes with a critical discussion of the design trade-offs. By exploring alternative architectural configurations that were ultimately discarded, this section demonstrates that the chosen design represents an optimal balance, ensuring that no obvious modifications could yield a strict improvement in one security property without causing a degradation in another.

3.2 Property-by-property verification

Integrity

Definition: reviews and reputation scores cannot be altered or deleted without detection.

Implementation: the system achieves this by decoupling computational logic from storage. Review metadata and scores are managed via the immutable state of the EVM smart contracts, while the actual text and multimedia payloads are stored on IPFS. The IPFS Content Identifier (CID) is permanently anchored to the blockchain through event

logs.

Security Argument: if an adversary $\mathcal{A}$ (e.g., a malicious storage node) attempts to alter the content of a published review, the IPFS protocol will deterministically compute a completely different CID. To successfully violate system integrity, $\mathcal{A}$ must either find a collision in the underlying cryptographic hash function (SHA-256) used by IPFS or compromise the consensus mechanism of the public Ethereum blockchain to rewrite the historical event logs. Both operations are computationally infeasible.

Authenticity and Sybil Resistance

Definition: every participant represents a unique verified entity; fake identities cannot be mass-created.

Implementation: this property is enforced through the strict requirement of a valid Proof of Purchase (PoP) for participation, combined with the issuance of non-transferable Soulbound Tokens (SBTs) for identity and reputation tracking.

Security Argument: if an adversary $\mathcal{A}$ attempts a Sybil attack by generating multiple wallet addresses to manipulate consensus, $\mathcal{A}$ must acquire a distinct, valid PoP for each identity. Consequently, $\mathcal{A}$ is forced to incur the real-world economic cost of purchasing physical PC components for every fake node. Furthermore, because SBTs are strictly non-transferable, $\mathcal{A}$ cannot pool the reputation farmed by multiple fake identities into a single master account, rendering the attack economically irrational.

Entitlement Correctness

Definition: reviews can only be submitted by users who actually purchased the specific component.

Implementation: the smart contract enforces this during the review upload phase through atomic verifications (Checks 1 and 2 of 2.3.1). It validates the SD-JWT signature, ensuring it matches the authorized E-commerce platform's public key, and verifies that the submitting address corresponds to the credential's declared subject.

Security Argument: if a malicious user or a dishonest seller $\mathcal{A}$ attempts to submit a review without a valid purchase, $\mathcal{A}$ must forge the E-commerce platform's digital signature over the SD-JWT credential. Forging this cryptographically robust entitlement proof is impossible without the platform's private key.

Double-Submission Prevention

Definition: one purchase grants the right to submit exactly one review; proofs of purchase cannot be reused.

Implementation: the protocol implements an on-chain registry of spent nullifiers. During checkout, the issuer computes a deterministic `reviewNullifier` representing

the unique transaction. Upon successful review submission, the smart contract cross-references this nullifier (Check 3 of 2.3.1) and permanently records it in a state mapping, marking the credential as spent.

Security Argument: if an adversary $\mathcal{A}$ attempts a replay attack to submit a second review using a previously validated PoP, the smart contract's uniqueness constraint will systematically revert the transaction. To violate this property, $\mathcal{A}$ would need to bypass the deterministic state transition logic of the Ethereum Virtual Machine, which is mathematically enforced by the network's consensus.

Censorship Resistance

Definition: legitimate reviews cannot be blocked, hidden, or removed by the seller or the platform.

Implementation: the architecture completely decouples the E-commerce platform (acting strictly as the transaction verifier) from the reputation layer (hosted on a permissionless blockchain).

Security Argument: if a curious platform or a dishonest seller $\mathcal{A}$ wishes to censor a negative review, they cannot simply delete it from a centralized database. To hide the review, $\mathcal{A}$ would have to either censor the user's transaction at the blockchain network level (requiring control over a significant portion of Ethereum's block builders) or execute a global coordinated unpinning attack on IPFS to destroy data availability, neither of which is achievable by a single commercial entity.

Pseudonymity

Definition: users can prove purchase entitlement without revealing personal information.

Implementation: the system employs the SD-JWT standard to enable selective disclosure. During interaction with the smart contract, the user reveals only the `userID`, `productID`, `vendorID`, and `reviewNullifier`, while keeping sensitive data (e.g., shipping address, full name) hidden inside cryptographic commitments.

Security Argument: if a general public observer $\mathcal{A}$ attempts to de-anonymize a reviewer by extracting personal information from the blockchain, $\mathcal{A}$ would need to reverse the cryptographic commitments contained within the `_sd` array of the SD-JWT or reverse-engineer the `Secret_Salt` hashed within the nullifier. Under standard cryptographic assumptions, these one-way functions cannot be reversed.

Observer	Pseudonymity guaranteed?	Rationale
General public / on-chain observer	Yes	Only the wallet address (0xABCD. . .) is visible on-chain. No real-world identity is inferable.
Third-party platforms (other participating e-commerce platforms)	Yes	They have no access to the Web2 account = wallet address mapping held by the issuing platform.
Issuing e-commerce platform	No (accepted trade-off)	The platform learns W_u at check-out and can therefore correlate blockchain activity with its internal user database.

Table 3.1: Pseudonymity guarantees by observer class.

Metric Integrity and Algorithmic Transparency

Definition: reputation scores reflect genuine community consensus, and the visibility of feedback is governed by transparent, non-manipulable criteria rather than proprietary sorting algorithms.

Implementation: the system computes a public **visibility_score** for each review by excluding negative-utility feedback and scaling voting influence logarithmically ($W = \log(1 + \mathcal{R}_{\text{voter}})$). Crucially, the dApp frontend sorts reviews strictly based on this deterministic on-chain value, without involving any off-chain ranking logic. Reputation updates rely on an asymmetric risk mechanism (Δ^- vs Δ^+).

Security Argument: if an adversary $\mathcal{A}$ (e.g., an e-commerce operator) wishes to shadow-ban a negative review or artificially boost a sponsored one, they cannot simply tweak a centralized sorting algorithm. The ranking is deterministically tied to the immutable blockchain state. Furthermore, if a farming attacker attempts to maximize token rewards, the asymmetric penalty (Δ^-) yields a negative expected value for random voting, requiring an impossible perfect consensus prediction oracle to succeed.

Security Property	Implementation Mechanism	Security Argument / Barrier
Integrity	EVM state constraints combined with IPFS CIDs anchored via on-chain event logs.	Requires finding a SHA-256 hash collision or executing a 51% attack to rewrite the Ethereum ledger.
Authenticity and Sybil Resistance	Mandatory Proof of Purchase (PoP) requirement and issuance of non-transferable Soulbound Tokens (SBTs).	Forces the adversary to incur a real-world economic cost (purchasing hardware) for every generated fake identity.
Entitlement Correctness	Smart contract validation of the SD-JWT credential via the <code>ecrecover</code> precompile (Checks 1 and 2 of 2.3.1).	Requires forging the issuing E-commerce platform's robust ECDSA cryptographic signature.
Double-Submission Prevention	Cross-referencing and permanently storing the <code>reviewNullifier</code> in an on-chain state mapping (Check 3 of 2.3.1).	Requires bypassing the deterministic state transition constraints enforced by the Ethereum Virtual Machine.
Censorship Resistance	Complete architectural decoupling of the E-commerce platform from the permissionless reputation ledger.	Requires network-level transaction censorship by block builders or a globally coordinated IPFS unpinning attack.
Pseudonymity	SD-JWT selective disclosure mechanism, revealing only essential claims (<code>userDID</code> , <code>productID</code> , <code>vendorDID</code> , <code>reviewNullifier</code>) to the smart contract.	Requires reversing cryptographic commitments or reverse-engineering the <code>Secret_Salt</code> hashed within the nullifier.
Metric Integrity and Transparency	Exclusion of negative-utility reviews, asymmetric reputation updates ($\Delta^- > \Delta^+$), and deterministic on-chain review sorting.	Random voting yields a negative expected value; manipulating the display ranking requires altering the immutable blockchain state rather than tweaking a Web2 algorithm.

Table 3.2: Summary of property-by-property verification and adversary barriers.

3.3 Threat-by-threat analysis

For each adversary introduced in Section 1.5, we walk through the corresponding attack surface as exposed by the architecture of WP2, identifying which residual risks remain and, where relevant, which behaviors are deliberately left unconstrained because they do not violate any security property. The cryptographic and economic barriers that neutralize each mechanism have already been established property-by-property in 3.2; here we do not re-derive them, but trace how an adversary would actually attempt to

move through the system, and focus on what 3.2 does not already cover.

Malicious user

The two primary attack vectors executed by an uncoordinated or automated malicious user, as formalized in Section 1.5, involve Sybil identity flooding and credential replay. The architecture neutralizes these threats through distinct cryptographic and structural barriers:

Defense against Sybil Fake buyers. If an adversary generates numerous pseudonymous Ethereum wallet addresses to simulate a crowd of fake buyers and manipulate product display rankings, the attack is intercepted at the smart contract validation layer. Because every review submission and utility vote programmatically requires the presentation of a valid Proof of Purchase signed by a trusted e-commerce issuer, the adversary cannot generate participating identities for free. Minting N participating wallets requires financing N legitimate, physical hardware purchases. This strict linear economic friction renders Sybil flooding financially prohibitive.

Defense against Proof of Purchase replay. An adversary attempting to reuse a single valid Proof of Purchase across multiple transactions to submit duplicate reviews or cast multiple utility votes for the exact same transaction is blocked by on-chain state constraints:

- **At the review layer:** During the first valid review submission, the smart contract extracts the deterministic digest (**reviewNullifier**) and permanently records it in an on-chain state mapping. Any subsequent attempt to submit a review using the same purchase credential inevitably reverts, as the contract detects that the nullifier has already been spent.
- **At the voting layer:** To prevent double-voting during the 30-day curation window, the contract checks an on-chain registry that permanently links the voter's address to the target review identifier. Any attempt by the same wallet or credential to cast a second utility vote on the same review is systematically rejected.

What remains possible, by design. A user who genuinely purchases the exact same hardware component multiple times on separate occasions—or orders multiple distinct items—receives a distinct Proof of Purchase (featuring a unique order ID and cryptographic salt) for each verified checkout.

The system legitimately permits this user to submit one review and cast one utility vote per valid credential. This is explicitly not classified as a security vulnerability: the *Authenticity and Sybil Resistance* property bounds system participation to verifiable economic stake, not to an arbitrary quota per human identity. Because each interaction corresponds to real commercial activity, the accumulated influence remains entirely legitimate. Crucially, even if a wealthy user completes numerous purchases, their capacity to dominate the curation layer is strictly bounded by the logarithmic compression of voting weight ($W = \log(1 + \mathcal{R})$) and the requirement to consistently align with community consensus.

Dishonest seller (bribery and self-dealing)

Self-dealing. Self-dealing inherits the same linear economic cost discussed in 3.3 for Sybil attacks: each fake review still requires an actual purchase, a delivery, and a freshly consumed nullifier. The buyer's and seller's on-chain identities are, by design, cryptographically unlinked at the protocol layer to preserve pseudonymity (Section 1.6), so self-dealing is technically feasible and not detectable by the general public. What is new here is that the issuing platform, which does observe real shipping and payment metadata under the honest-but-curious assumption 3.3, is in a privileged position to detect anomalous patterns, such as a seller's own payment instrument recurring behind purchases of its own catalog.

Off-chain bribery. While ledger cryptography cannot enforce human sincerity (an accepted boundary of Entitlement Correctness), the architecture neutralizes bribery economically through community curation.

If a bribed user submits an overly positive review that contradicts genuine product quality, the community will cast downvotes. Once the review's algorithmic visibility drops below zero, the protocol automatically excludes it from the seller's aggregate reputation score.

Consequently, the vendor absorbs the financial cost of the bribe without achieving any reputational gain.

Onboarding authentication replay (Vendor Impersonation). While bribery and self-dealing involve a legitimately registered vendor acting maliciously, the architecture must also protect against external adversaries attempting to hijack a vendor's identity.

An adversary might intercept the authentication payload sent by a legitimate vendor to the ECommerce Service and attempt to replay it to illegitimately complete the registration process.

To mitigate this impersonation vector, the ECommerce Service implements a strict challenge-response protocol: prior to registering the vendor, it generates a cryptographic **nonce** that the vendor must sign using the private key associated with their DID. The service then resolves the **vendorDID** on-chain to retrieve the vendor's public key and validates the signature, guaranteeing request freshness, proving active key possession, and ensuring historical payloads cannot be reused.

Curious platform (honest-but-curious issuer)

Table 3.1 and the cryptographic argument of 3.2 (Pseudonymity) already establish *that* pseudonymity holds for every observer class except the issuing e-commerce platform. Here we motivate *why* this asymmetry is structurally irreducible within the issuer-centric trust model adopted in Section 1.4, rather than an oversight.

The platform is the sole trusted Issuer of the PoP (Section 1.3): to legitimately certify that a specific real-world buyer completed a specific real-world transaction, it must, during checkout, learn which wallet address W_{user} the credential subject binds to. Entitlement Correctness (Section 1.6), the very property that makes reviews trustworthy in the first place, depends on this binding being performed honestly by a party that has independent, off-chain knowledge of the transaction. It is therefore not possible for the

issuer to simultaneously (a) authoritatively vouch for a real-world purchase and (b) be cryptographically blind to which wallet it is vouching for.

The honest-but-curious assumption bounds the practical risk rather than eliminating it cryptographically: the platform is assumed not to actively exploit or publish this correlation. Crucially, this residual risk is confined to a single point of trust rather than propagated system-wide, unlike centralized reputation systems, where the review identity correlation is the default visible to the operator, every other participant in the ecosystem (competing platforms, the general public, other users) never observes it, which is itself a meaningful containment of the threat.

Apathy (curator apathy)

Low voluntary participation in utility voting directly degrades the statistical robustness of the weighted consensus score C (Section 2.5). In a decentralized curation model, widespread voter apathy shrinks the active sample size, allowing a small, coordinated subset of participants to disproportionately distort the final consensus outcome, the resulting `visibility_score`, and, transitively, the aggregated seller and product reputation metrics ($\mathcal{R}_{\text{vendor}}$ and $\mathcal{R}_{\text{product}}$).

Mitigation via Economic Incentives (ReputationTokens). To mitigate this metric-integrity risk and actively discourage abstention, the architecture incorporates a tangible cryptoeconomic incentive structure. Rather than relying purely on altruistic community participation, the protocol rewards active, consensus-aligned curators with **ReputationTokens** implemented as non-transferable Soulbound Tokens (SBTs). Crucially, because these tokens hold direct purchasing utility via the **Burn-to-Redeem** mechanism (allowing users to burn accumulated tokens in exchange for real-world fiat discounts on future e-commerce purchases) participants have a compelling financial incentive to overcome apathy, participate in the 30-day curation windows, and contribute to the statistical resilience of the consensus mechanism.

Farming (incentive manipulation)

Mitigating random voting via asymmetric risk. Because the protocol enforces an asymmetric reputation penalty that strictly outweighs the reward ($\Delta^- > \Delta^+$), voting at random yields a mathematically negative expected return over time. Consequently, an adversary attempting to farm rewards through random guessing will systematically deplete their reputation score ($\mathcal{R}$). Because effective voting influence is scaled logarithmically ($W = \log(1 + \mathcal{R})$) and token minting is directly proportional to current reputation, a random farmer rapidly loses both curation influence and token-earning capacity, rendering the strategy self-defeating.

Last-mover free-riding (Accepted residual risk). However, because running upvote and downvote counters (U and D) remain publicly readable from smart contract state throughout the 30-day window, a rational farmer is not limited to random guessing. An adversary can adopt a last-mover strategy: waiting until shortly before window closure to cast a vote that blindly aligns with the visible majority, thereby securing a reward without genuinely evaluating review utility.

While this strategy still requires a valid Proof of Purchase (preventing zero-cost execution

at scale), it undermines the independent judgment rationale of the consensus. We explicitly recognize this open-state transparency as an accepted residual limitation of the current design, noting that it can be structurally resolved in future iterations by transitioning to a cryptographic **Commit-Reveal voting scheme**.

3.4 Design trade-offs and optimality

A fundamental requirement of robust system verification is demonstrating that the architecture occupies an optimal point within the design space. In decentralized systems, the “no free lunch” theorem practically dictates that it is rarely possible to strictly improve one security property without degrading another.

To validate the optimality of the proposed solution (WP2), we explicitly analyze three alternative architectural modifications that were considered during the design phase and subsequently discarded. For each alternative, we highlight the inherent trade-off, proving that the current design choices represent the most balanced compromise between security, efficiency, and real-world viability.

Trade-off 1: Data Availability vs. Economic Sustainability

Alternative considered Store the full text and multimedia content of the reviews directly within the smart contract state (on-chain), rather than relying on the InterPlanetary File System (IPFS).

Perceived benefit This would strictly improve the system’s *Data Availability*. The persistence of the reviews would be mathematically guaranteed by the Ethereum consensus layer, completely eliminating the reliance on consortium nodes for off-chain data pinning.

The cost (Why it was discarded) Writing large arbitrary strings to the EVM state is prohibitively expensive due to gas costs. Implementing this modification would destroy the economic sustainability of the platform, pricing out average users and effectively halting adoption. The hybrid approach (off-chain storage with on-chain CID event logging) is a necessary and optimal compromise: it sacrifices absolute on-chain availability to achieve economic viability, while fully preserving cryptographic *Integrity*.

Trade-off 2: Participation Volume vs. Sybil Resistance

Alternative considered Allow any registered user to cast utility votes on reviews, removing the strict requirement of holding a matching Proof of Purchase (PoP) for the curation layer.

Perceived benefit This would drastically lower the barrier to entry for voters, generating a massive increase in participation and effectively neutralizing the *Apathy* threat analyzed in Section 3.3.

The cost (Why it was discarded) Removing the PoP requirement at the voting stage would completely strip the curation layer of its economic barrier. An adversary could generate thousands of zero-cost fake identities to artificially upvote their own reviews or downvote competitors. This would yield a catastrophic failure of *Authenticity and Sybil*

Resistance. The architecture deliberately accepts lower turnout (apathy) as a necessary cost to ensure that every single vote is backed by verifiable economic stake.

Trade-off 3: Resolution Speed vs. Metric Integrity

Alternative considered Eliminate the 30-day curation window, allowing the `visibility_score` and the vendor's reputation to update instantly upon every new vote, and immediately distributing token rewards.

Perceived benefit This would provide immediate gratification for curators and real-time reputation feedback, significantly improving the user experience and the responsiveness of the platform.

The cost (Why it was discarded) Instant finalization removes the time required for a statistically significant and diverse consensus to form. It would leave the system highly vulnerable to short-term, coordinated manipulation campaigns (e.g., a flash mob of coordinated votes overriding the organic consensus before the broader community has a chance to react). The mandatory 30-day window is a deliberate friction point: it sacrifices speed to guarantee robust *Metric Integrity* and to provide sufficient time for the asymmetric penalty mechanism (Δ^- vs Δ^+) to organically filter out bad actors.

3.5 Residual Limitations and Future Work

As demonstrated in the preceding analysis, the proposed architecture effectively mitigates the primary threats outlined in the system model. However, to maintain rigorous analytical integrity, we explicitly acknowledge the structural boundaries of the current design. The following residual limitations represent accepted trade-offs within our trust model, providing concrete directions for future architectural iterations:

Off-Chain Bribery and Collusion

As established in the threat-by-threat analysis, while ledger cryptography cannot enforce human sincerity, the architecture economically neutralizes blatant bribery through community curation: reviews that contradict genuine product quality attract net downvotes ($\sum U - \sum D < 0$) and are automatically excluded from the seller's aggregate reputation score ($\mathcal{R}_{\text{vendor}}$). Nevertheless, subtle off-chain bribery—where a vendor financially compensates a consumer to submit a plausible positive review that avoids community downvotes, or incentivizes the removal of a legitimate negative review—remains outside the syntactic reach of EVM smart contracts.

Because the author holds a valid Proof of Purchase and signs with their legitimate private key, the state machine cannot detect the external off-chain inducement.

Future work could explore integrating decentralized oracle networks (e.g., Chainlink) to feed statistical sentiment-shift analysis and refund-correlation monitoring directly into contract scoring logic.

Issuer-Level Linkability

The system guarantees pseudonymity against the general public and competing marketplaces. However, as analyzed under the honest-but-curious assumption, the requirement that a trusted e-commerce platform issues the Proof of Purchase inherently creates a single point of residual linkability. To authoritatively certify a real-world checkout, the issuing platform must necessarily bind the commercial order details to the consumer's Decentralized Identifier (**userDID**) in its private database. While cross-platform wallet isolation prevents this risk from propagating system-wide, completely eliminating issuer linkability would require transitioning from standard W3C SD-JWTs to Zero-Knowledge Proofs of Purchase (ZK-PoP, utilizing ZK-SNARKs or ZK-STARKs). In such a future iteration, a consumer could prove on-chain that they hold a valid credential signed by the marketplace without revealing their specific DID, wallet address, or order reference to the issuer during the review submission transaction.

Last-Mover Free-Riding in Curation

The asymmetric reputation update mechanism ($\Delta^- > \Delta^+$) successfully eliminates random reward farming by enforcing a mathematically negative expected return. However, because running upvote and downvote counters (U and D) remain transparently readable in open-state EVM storage throughout the 30-day curation window, a rational adversary can adopt a last-mover strategy: waiting until shortly before window closure to cast a vote that blindly copies the visible majority. While this strategy still requires a valid Proof of Purchase (preventing zero-cost Sybil execution), it allows legitimate credential holders to secure Soulbound ReputationTokens without genuinely evaluating review utility. To structurally eliminate this free-riding vector, a future iteration of the protocol should implement a two-phase cryptographic Commit-Reveal voting scheme: during the active 30-day window, utility votes are hash-committed in secret ($\text{keccak256}(V_i + \text{salt})$), and only revealed and tallied after the curation period formally closes.

3.5.1 Results Recap

To consolidate the findings of the detailed threat-by-threat analysis, Table 3.3 provides a schematic overview of the adversarial model. It categorizes the specific attack vectors explored throughout this section and directly pairs them with the corresponding architectural mechanisms designed to neutralize them. Furthermore, the summary explicitly delineates the boundaries of the system's security guarantees by highlighting the accepted trade-offs and residual limitations inherent to the current design, which ultimately serve as the foundation for future iterations.

Threat Category	Specific Attack Vector	Proposed Mitigation / System Response
Malicious User	Sybil Fake Buyers (Identity flooding)	Mandatory Proof of Purchase (PoP) imposes a strict linear economic cost (hardware purchase) per identity.
	PoP Replay (Duplicate reviews/votes)	EVM state constraints. The reviewNullifier is permanently burned; on-chain mappings prevent double-voting.
Dishonest Seller	Self-dealing (Buying own products)	Inherits the linear economic cost of real purchases. Anomalous payment patterns are detectable by the issuing platform off-chain.
	Off-chain Bribery	Community curation. Bribed positive reviews are downvoted, dropping their visibility below zero and neutralizing any reputational gain.
	Vendor Impersonation (Auth replay)	Challenge-response protocol using a cryptographic nonce signed by the vendor's DID private key.
Curious Platform	Off-chain Identity Linkability	Accepted Trade-off: The issuer inherently learns the Web2-to-Wallet mapping at checkout. The risk is isolated to a single entity, preserving system-wide pseudonymity.
Community Apathy	Low participation distorting consensus	Cryptoeconomic incentives via non-transferable Soulbound ReputationTokens featuring a Burn-to-Redeem mechanism (flat discounts).
Reward Farming	Random Voting	Asymmetric penalty ($\Delta^- > \Delta^+$) yields a mathematically negative expected return for random guessing.
	Last-mover free-riding (Copying majority)	Residual Limitation: Mitigated by the PoP cost, but structurally requires a Commit-Reveal scheme (planned for future iterations) to hide live vote tallies.

Table 3.3: Summary of identified threat vectors, architectural mitigations, and accepted residual risks.

Workpackage 4

Implementation and Evaluation

Source code available at: <https://github.com/mich4l3/ReviewChainDApp>

This chapter bridges the theoretical foundation established in the previous workpackages with a concrete, functional prototype. While Workpackage 2 defined the architectural blueprint of the decentralized reputation system and Workpackage 3 rigorously analyzed its security guarantees, Workpackage 4 presents the actual implementation and evaluation of the resulting Proof of Concept (PoC).

The primary objective is to demonstrate how the conceptual hybrid Web2.5 architecture has been translated into deployable, testable code that preserves the security and privacy invariants formalized in the system model. The discussion is organized around the three macro-components of the ecosystem:

- **The Smart Contract Layer:** six modular Solidity contracts deployed on an EVM-compatible local network, implementing the on-chain identity registries, the DID resolution, the review lifecycle state machine, the token-weighted curation mechanism, and the Soulbound incentive token.
- **The Issuer Backend:** a lightweight Node.js microservice simulating the e-commerce platform as the “honest-but-curious” trust anchor. It issues gas-optimized raw ECDSA Proof-of-Purchase credentials whose layout is byte-identical to the on-chain verification hash.
- **The Decentralized Application Frontend:** a purely client-side interface (Vanilla JS, HTML5, CSS3) that orchestrates wallet authentication, automatic DID registration, off-chain IPFS storage, and direct on-chain transaction construction without requiring any centralized application server.

Finally, Section 4.5 presents a comprehensive integration test suite of 26 automated test cases that exercise every execution path defined in the protocol specification, including time-dependent curation mechanics validated through controlled blockchain time-travel and adversarial conditions to prove system resilience.

4.1 Technology Stack

Architectural Layer	Adopted Technology	Role in the Proof of Concept
User Identity	Ethers.js v6 / MetaMask	Pseudonymous authentication mapped to W3C Decentralized Identifiers (DIDs).
Issuer Backend	Node.js / Express.js	Lightweight Web2 microservice for verifying transactions and generating SD-JWT payloads.
Smart Contract Layer	Solidity 0.8.24 / Hardhat 3	EVM execution of on-chain business logic (registries, review lifecycle, utility voting).
Proof of Purchase	Raw ECDSA (ES256K)	Gas-optimized signatures leveraging EVM-native <code>ecrecover</code> , bypassing EIP-191 prefixes.
Frontend dApp	Vanilla JS, HTML5, CSS3	Static, lightweight client-side interface orchestrating the Web3 submission pipeline.
Content Storage	IPFS Simulation (Web Crypto API)	Off-chain review storage simulated via deterministic SHA-256 hashing to generate CIDs.
Mathematical Library	Solady FixedPointMathLib	Gas-optimized <code>lnWad</code> for computing the logarithmic voting weight $W = \ln(1 + R)$.
Incentive Layer	Custom ERC-20 (Soul-bound)	Non-transferable Reputation Tokens (RWT) minted to reward consensus-aligned behavior.

Table 4.1: Summary of the technology stack utilized for the Proof of Concept implementation.

To realize the hybrid Web2.5 architecture proposed in this project, the technology stack was selected to ensure a clear separation of concerns between the centralized trust anchor (the e-commerce issuer) and the decentralized execution environment (the blockchain and the client). The implementation prioritizes lightweight, open-source standards to demonstrate the feasibility of the protocol without relying on proprietary frameworks.

4.1.1 Blockchain Layer (Smart Contracts and EVM)

The on-chain business logic was implemented in **Solidity 0.8.24**, the industry-standard language for EVM smart contracts. For development, testing, and deployment, the project employs **Hardhat 3** with its local node (operating on `chainId 31337`). This isolated environment provides deterministic transaction mining and programmatic control over blockchain state variables, most critically, the block timestamp, enabling the simulation of the 30-day curation window without incurring gas costs on a public testnet.

For the logarithmic voting weight function $W = \ln(1 + R)$ specified in WP2 2.5, the implementation delegates to **Solady's FixedPointMathLib.lnWad**, a widely audited, gas-optimized library that computes natural logarithms directly in 10^{18} ("WAD") fixed-point arithmetic. This avoids the need for a hand-rolled approximation and guarantees the strictly monotonic property that the anti-farming argument in WP3 3.3 depends upon.

4.1.2 Backend Issuer (Web2 E-Commerce Simulation)

The centralized entity responsible for issuing the cryptographic Proof of Purchase was built using **Node.js** and the **Express.js** web framework. This stack was chosen for its asynchronous, event-driven I/O model. Since the issuer's sole responsibility is to expose an API endpoint (**POST /request-pop**) that computes hashes and generates raw ECDSA signatures, a minimal Node.js server encapsulates this role without the overhead of a full backend framework.

4.1.3 Frontend Decentralized Application

The user-facing dApp was developed using **Vanilla JavaScript**, **HTML5**, and **CSS3**, deliberately avoiding heavy frameworks such as React or Next.js. This architectural choice demonstrates that the dApp operates as a purely static, lightweight client. By relying exclusively on standard Web APIs and loading the **Ethers.js v6** library via CDN, the frontend can theoretically be hosted on decentralized networks (such as IPFS itself) in a production deployment, ensuring that no centralized server is required to serve the application logic.

4.1.4 Web3 Integration and Decentralized Storage

The **Ethers.js v6** library handles all RPC communication between the client and the EVM, interfacing with the user's **MetaMask** wallet for identity management and transaction signing.

For off-chain content storage, the protocol relies on the **InterPlanetary File System (IPFS)**. In this Proof of Concept, the upload phase is simulated locally using the Web Crypto API to generate deterministic SHA-256 hashes. This accurately replicates the creation of a Content Identifier (CID) that the smart contract stores as the review's canonical reference, without requiring an active connection to the global IPFS peer-to-peer network. Within a single browser session, the simulated IPFS layer also supports content retrieval: the frontend maintains an in-memory map from CID to plaintext, enabling the user interface to display previously submitted reviews.

4.2 Smart Contract Implementation

The on-chain core of the system is composed of a modular suite of six Solidity contracts plus one library. This architecture intentionally decouples state storage from business logic, ensuring that access control boundaries are strictly defined and attack surfaces are minimized.

4.2.1 Core Contracts Architecture

DIDRegistry. Anchors the W3C standard `did:ethr` Decentralized Identifiers natively on-chain. It resolves string identifiers (e.g., `did:ethr:0x123...`) to their underlying Ethereum address owner and maintains an active flag for each identity, structurally enforcing liveness. In addition to the standard self-registration path (`registerDID`), the contract exposes `issuerRegisterDID`, which allows an authorized E-Commerce platform (verified against `IdentityRegistry`) to register a DID on behalf of a third-party address. This supports vendor onboarding flows in which the issuer anchors the vendor's DID Document on-chain without requiring the vendor to submit the registration transaction themselves.

IdentityRegistry. This contract anchors the public keys of the two off-chain trust roots that the protocol requires: Certification Authorities (CAs), who vet and vouch for vendors (WP2 2.2.1), and E-commerce platforms (Issuers), who sign the Proof of Purchase (WP2 2.3). The contract exposes `isCA(address)` and `isIssuer(address)` mappings, checked downstream by `VendorRegistry` and `ReviewContract` respectively. The constructor accepts optional `genesisCA` and `genesisIssuer` parameters to pre-register the initial trust roots at deployment time.

VendorRegistry. Implements Seller Onboarding (WP2 2.2.1). The contract exposes the `issuerRegisterVendor` function, allowing an authorized E-Commerce platform to directly register a vendor on-chain after successfully verifying their identity off-chain. This maintains the mapping of verified vendor wallets and their corresponding legal identities without requiring the vendor to submit a complex Verifiable Credential transaction directly to the EVM, optimizing gas costs and aligning with the issuer-delegated validation flow.

NullifierRegistry. Dedicated exclusively to preventing replay attacks and the double-spending of reviews. When a valid Proof of Purchase is submitted, its unique `nullifier` is permanently recorded via the `spend()` function, which reverts with `AlreadySpent` on any reuse attempt. Write access is restricted to the `ReviewContract` through the `onlyReviewContract` modifier.

ReputationToken (Soulbound Token). Implements the protocol's economic incentive layer. The contract maintains an ERC-20-compatible `balanceOf` interface for wallet compatibility, but `transfer()`, `transferFrom()`, and `approve()` are overridden to *unconditionally revert* with the custom error `SoulboundTokenNonTransferable`:

```
function transfer(address, uint256) external pure returns (bool) {
    revert SoulboundTokenNonTransferable();
}

function transferFrom(address, address, uint256)
external pure returns (bool) {
    revert SoulboundTokenNonTransferable();
}
```

```
function approve(address, uint256) external pure returns (bool){
    revert SoulboundTokenNonTransferable();
}
```

This transforms the asset into a strictly non-transferable Soulbound Token (SBT), ensuring that reputation and voting influence cannot be traded or pooled on secondary markets. Minting and burning privileges are exclusively restricted to the **ReviewContract**.

FixedPointMath (Library). Wraps Solady’s **FixedPointMathLib.lnWad** into a single-function library that computes $\ln(1 + R)$ in WAD (10^{18}) fixed-point, serving as the voting weight function W of WP2 2.5.

ReviewContract. The central orchestrator of the protocol, discussed in detail in the following subsections.

4.2.2 The Review Struct and On-Chain State

The **ReviewContract** manages the entire review lifecycle through a storage struct that captures both the submission metadata and the curation state:

```
struct Review {
    address reviewer;
    bytes32 productIdHash;
    address vendor;           // resolved from DIDRegistry via
    ↪ vendorDID
    string cid;               // current IPFS CID
    uint8 score;              // 1-5
    uint64 submittedAt;
    uint64 windowStart;       // resets on modification
    bool modified;
    bool revoked;
    bool curationClosed;
    bool includedInAggregation;
    uint256 upvoteWeight;     // fixed-point 1e18
    uint256 downvoteWeight;   // fixed-point 1e18
    uint256 reputationAtSubmission; //  $W_i$  for aggregation
}
```

The struct stores only the CID, not the review text, on-chain, enforcing data minimization and reducing gas costs. The full review content resides off-chain on IPFS and remains verifiable through its content-addressed hash.

Two timestamps are tracked separately. **submittedAt** records the original submission moment and never changes. **windowStart** is initialized to **submittedAt** but is reset to **block.timestamp** whenever a modification is made: because a modification re-introduces the review into the curation queue as entirely new content (resetting votes

and re-opening the curation period), it opens a fresh 30-day revocation window as well. Revocation is gated on `windowStart`, not on `submittedAt`, which is the intended behavior.

4.2.3 On-Chain Signature Verification

Users interact directly with the `ReviewContract` via their wallet. The verification of the Proof of Purchase, executed inside the `submitReview` and `voteOnReview` functions, leverages EVM-native cryptographic mechanisms:

```
// 1. Resolve DID and ensure it is active
DIDDocument memory doc = didRegistry.resolveDID(reviewerDID);
if (!doc.active) revert DIDNotActive();
address reviewer = doc.owner;

// 2. Ensure the caller owns the DID
if (reviewer != msg.sender) revert NotReviewer();

// 3. Recompute the canonical Proof of Purchase hash
bytes32 productIdHash = keccak256(bytes(productId));
bytes32 h = keccak256(
    abi.encode(reviewerDID, vendorDID, productIdHash, nullifier,
        ↪ sdDigests)
);

// 4. Verify ECDSA signature (SD-JWT Integrity)
address recovered = ecrecover(h, v, r, s);

// 5. Authenticate the Issuer
if (!identityRegistry.isIssuer(recovered)) revert UntrustedIssuer();
```

The execution flow explicitly implements the three checks defined in WP2 2.3:

1. **Wallet Binding (Check 2):** by dynamically resolving the DID through the `DIDRegistry`, the contract ensures that only the cryptographically proven owner of the DID (`msg.sender`) can submit the review, entirely preventing identity theft.
2. **SD-JWT Integrity (Check 1):** the contract recomputes the canonical hash precisely replicating the layout of the off-chain issuer. The native `ecrecover` extracts the signer's address. Any tampering with the payload silently alters the hash, causing `ecrecover` to yield a random address.
3. **Issuer Authentication (Check 1):** finally, the recovered address is verified against the `IdentityRegistry` to ensure it corresponds to an authorized E-commerce platform.

4.2.4 Curation Finalization and Reward Distribution

The `finalizeCuration` function implements the consensus determination, reputation update, and token reward distribution logic defined in WP2 2.5. It is callable by anyone after the 30-day curation window has elapsed:

```
function finalizeCuration(uint256 reviewId) external {
    Review storage rv = reviews[reviewId];
    if (rv.curationClosed) revert AlreadyFinalized();
    if (block.timestamp < rv.windowStart + CURATION_WINDOW)
        revert CurationWindowStillOpen();

    uint256 totalWeight = rv.upvoteWeight + rv.downvoteWeight;
    // C = U / (U + D), fixed-point 1e18
    uint256 consensus = totalWeight == 0
        ? 0
        : (rv.upvoteWeight * 1e18) / totalWeight;

    VoteRecord[] storage votes = _reviewVotes[reviewId];
    for (uint256 i = 0; i < votes.length; i++) {
        VoteRecord storage vr = votes[i];
        uint256 vi = vr.useful ? 1e18 : 0;
        uint256 diff = vi > consensus ? vi - consensus : consensus -
            ↪ vi;

        if (diff < thetaFixedPoint) {
            // Concordant voter: +Delta+, mint k * R * Delta+ tokens
            reputationScore[vr.voter] += deltaPlus;
            uint256 tokens =
                (deltaPlus * reputationScore[vr.voter] * kScaling) /
                ↪ 1e18;
            if (tokens > 0) reputationToken.mint(vr.voter, tokens);
        } else {
            // Discordant voter: -Delta- (no tokens)
            if (reputationScore[vr.voter] > deltaMinus)
                reputationScore[vr.voter] -= deltaMinus;
            else
                reputationScore[vr.voter] = 0;
        }
    }
    rv.curationClosed = true;

    // Reviews with non-negative visibility enter aggregation
    bool qualifies = !rv.revoked && rv.upvoteWeight >=
        ↪ rv.downvoteWeight;
```

```

if (qualifies) {
    _includeInAggregation(rv);
    // Reviewer reward (same formula as voter reward)
    uint256 reviewerTokens =
        (deltaPlus * reputationScore[rv.reviewer] * kScaling) /
        ↪ 1e18;
    if (reviewerTokens > 0)
        reputationToken.mint(rv.reviewer, reviewerTokens);
}
}

```

The function iterates over all vote records, computing the distance $|v_i - C|$ between each voter's stance and the aggregate consensus ratio. Voters within the threshold θ receive a reputation increase of Δ^+ and a token reward scaled by their updated reputation and the issuance factor k . Discordant voters lose Δ^- reputation points without receiving any tokens, implementing the asymmetric penalty ($\Delta^- > \Delta^+$) that WP3 3.3 identifies as the key anti-farming mechanism.

If the review's net visibility score is non-negative, it is included in the weighted reputation aggregation for both the product and the vendor, where:

$$R_{\text{product}} = \frac{\sum_i S_i \cdot W_i}{\sum_i W_i}$$

and S_i is the review score, W_i the reviewer's reputation at submission time.

4.2.5 Deployment Mechanics and Post-Deploy Wiring

The deployment sequence, implemented in `scripts/deploy.js`, addresses a circular dependency between the contracts. The `NullifierRegistry` and `ReputationToken` restrict their write operations (storing nullifiers, minting tokens) exclusively to the `ReviewContract` via `onlyReviewContract` modifiers. However, the `ReviewContract`'s constructor requires the addresses of these registries as immutable parameters.

To resolve this, the deployment script adopts a *post-deploy wiring* pattern: the registries are deployed first with open `reviewContract` slots, followed by the `ReviewContract` itself. A subsequent authorization transaction (`setReviewContract()`) cryptographically binds each registry to the newly deployed orchestrator. This function is guarded by `onlyOwner` and protected against re-initialization by the `AlreadyInitialized` check, ensuring that the binding is permanent and tamper-proof.

To prevent nonce synchronization issues during rapid sequential deployments on the auto-mining Hardhat node, the deployer's signer is wrapped with the Ethers.js `NonceManager`, which maintains the transaction nonce locally rather than querying the RPC endpoint between each deployment.

4.3 The Issuer Backend: Bridging Web2 and Web3

In the proposed architecture, the e-commerce platform operates under an “honest-but-curious” threat model: it is the sole trusted entity authorized to verify real-world purchases, yet it possesses no capability to alter or censor the resulting reviews on the blockchain. This separation of trust is implemented through a Node.js microservice that bridges the Web2 purchasing environment with the Web3 reputation ledger.

4.3.1 Proof of Purchase Generation and Selective Disclosure

The core functionality is encapsulated within the `POST /request-pop` endpoint. Upon confirming a successful commercial transaction, the user’s client requests a verifiable credential by providing the DID string (`userDID`) and the item’s SKU (`productId`). The server then computes:

1. **productIdHash**: `keccak256(bytes(productId))`, matching the on-chain representation.
2. **nullifier**: a deterministic hash linking the user and the product, computed as `keccak256(userDID || productId || SECRET_SALT)`. As specified in the WP2/WP3 threat model, this deterministic derivation is critical to enforce the “one review per purchase” invariant: the **NullifierRegistry** permanently marks it as spent upon first use, preventing even a compromised or malicious issuer from emitting multiple usable PoPs for the same real-world purchase.
3. **sdDigests**: an array of `keccak256` commitments over undisclosed claims (e.g., shipping address, payment amount). These digests are included in the signed payload to guarantee integrity without revealing the underlying data on-chain.

4.3.2 Cryptographic Compatibility: Raw ECDSA over secp256k1

A fundamental engineering challenge in bridging off-chain signatures with on-chain verification is ensuring byte-level encoding compatibility while minimizing gas consumption. Standard Ethereum wallets sign data using `personal_sign`, which automatically prepends the EIP-191 prefix (`"\x19Ethereum Signed Message\n32"`). Reconstructing and stripping this prefix on-chain would introduce unnecessary computational overhead.

To optimize the architecture, the backend deliberately bypasses EIP-191 in favor of a raw ECDSA signature over the `secp256k1` elliptic curve. The implementation proceeds as follows:

```
// 1. Encode payload identically to Solidity's abi.encode()
//   The vendorDID is included as the second field so the hash
//   covers the full tuple (userDID, vendorDID, productIdHash,
//   nullifier, sdDigests) – mirroring the on-chain ecrecover input.
const encoded = abiCoder.encode(
  ["string", "string", "bytes32", "bytes32", "bytes32[]"],
  [userDID, vendorDID, productIdHash, nullifier, sdDigests]
);
```

```
// 2. Hash with keccak256
const hash = ethers.keccak256(encoded);

// 3. Sign the raw digest (NO EIP-191 prefix)
//    signingKey.sign() != wallet.signMessage()
const sig = issuerWallet.signingKey.sign(hash);
```

The critical detail is at line 12: the server uses `signingKey.sign(hash)` rather than `wallet.signMessage()`, producing a signature over the raw keccak256 digest without any prefix. Because the payload is encoded with `ethers.AbiCoder`, which replicates the exact memory layout and padding generated by Solidity's `abi.encode()`, the on-chain `ecrecover` function recovers the correct issuer address without any additional processing. This design reduces the gas cost of each review submission by eliminating the on-chain prefix reconstruction entirely.

4.4 Decentralized Application (Frontend Demo)

The frontend orchestrates the complex interactions between the user, the Web2 issuer, and the Web3 blockchain within a seamless execution pipeline.

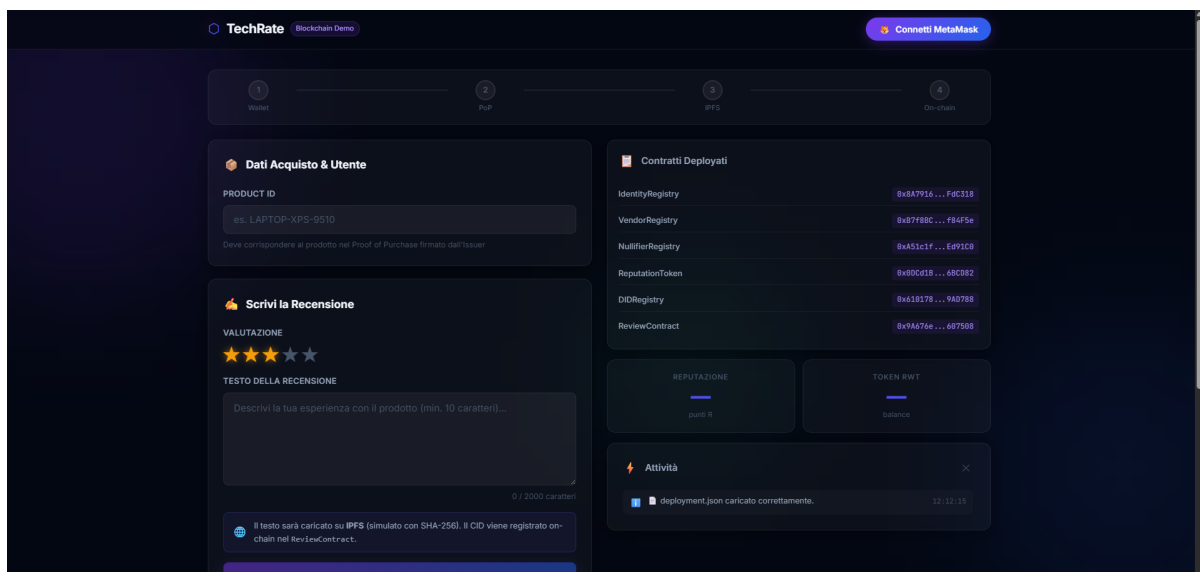


Figure 4.1: Initial window

4.4.1 Wallet Connection and DID Auto-Registration

The dApp authenticates users exclusively through their cryptocurrency wallets, interfacing with the `window.ethereum` provider injected by MetaMask. To ensure a frictionless onboarding process, the dApp performs an automatic background check against the `DIDRegistry` upon connection. If the user's wallet is not yet mapped to a Decentralized Identifier, the interface constructs the appropriate DID string (`did:ethr:0x...`) and prompts the user to sign an on-chain registration transaction via MetaMask. This

abstracts the entire Web3 identity creation layer away from the user, ensuring immediate readiness for protocol participation.

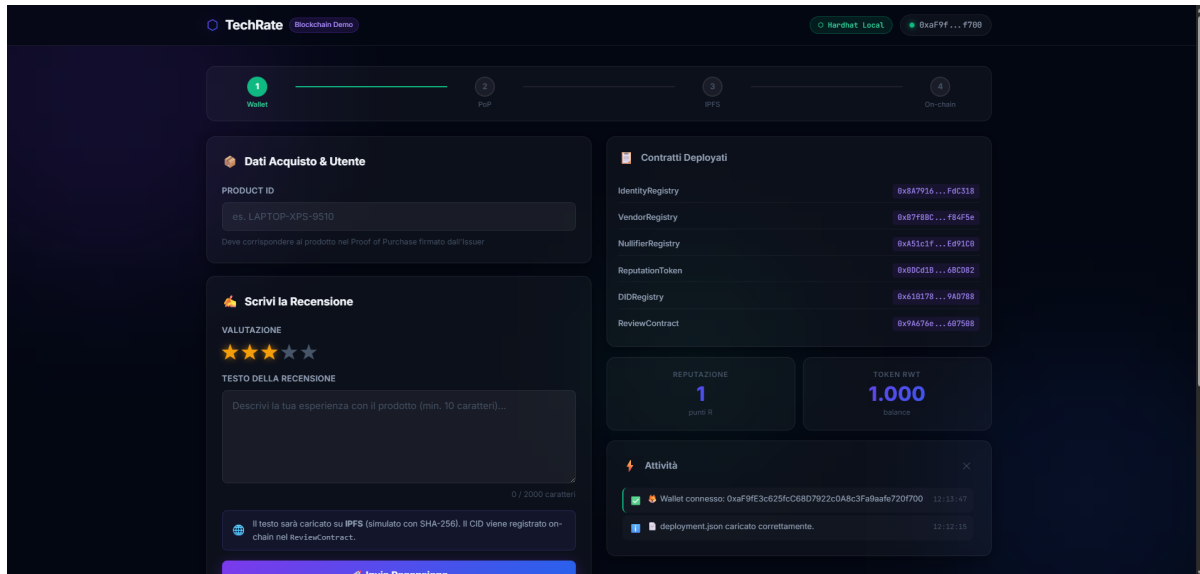


Figure 4.2: Wallet connection

4.4.2 The Submission Pipeline

When a user compiles a review through the UI, the frontend executes a highly orchestrated hybrid pipeline:

1. **Decentralized Storage (Off-chain):** the user writes the review text, which is processed through the simulated IPFS layer. The Web Crypto API computes a deterministic SHA-256 hash, producing a Content Identifier (CID). The text is cached locally in memory, while only the lightweight CID string proceeds in the pipeline, enforcing strictly optimal EVM data minimization.
2. **Proof of Purchase Request (Web2 Interaction):** the client transparently executes a `POST` request to the Issuer Backend at `/request-pop`, passing the DID and the product ID. The server validates the purchase off-chain and returns the ECDSA-signed cryptographic variables: `v`, `r`, `s`, the `nullifier`, and the `sdDigests`.
3. **Direct Submission (Web3 Interaction):** the dApp immediately packs all these parameters into a single payload and directly invokes `reviewContract.submitReview()`. MetaMask presents the transaction popup to the user. Once approved and mined, the review is immutably anchored to the ledger.

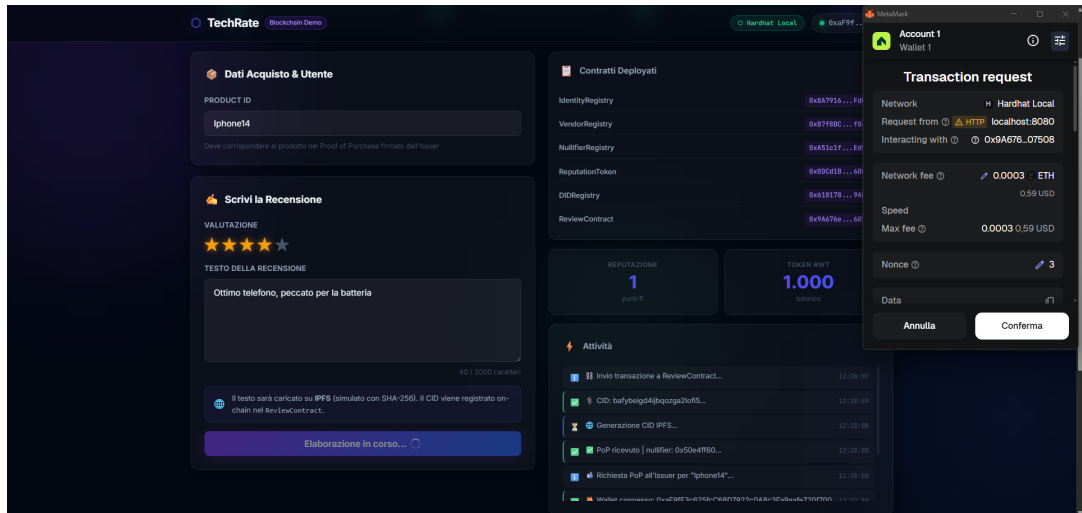


Figure 4.3: Review and Submitting

4.4.3 Event-Driven Responsiveness and Development Tooling

The frontend subscribes to specific Solidity events emitted by the blockchain (`ReviewSubmitted`, `CurationFinalized`, `ReviewerRewarded`, etc.) to dynamically update the interface in real time without manual polling, achieving a UX comparable to centralized applications.

To facilitate testing and evaluation, the UI includes a developer panel featuring a **Time Travel** mechanic. This button triggers a specialized `evm_increaseTime` RPC call to the local Hardhat node, artificially advancing the blockchain clock by 31 days. This empowers evaluators to instantly bypass the strict 30-day curation window and test the reward distribution logic without altering the production-ready smart contract code.

4.5 System Evaluation and Testing

To validate the theoretical models proposed in WP2 and the security guarantees analyzed in WP3, the implemented system was subjected to a comprehensive automated test suite. The suite was developed as a Hardhat 3 TypeScript integration test (`test/reviewchain.test.ts`) and executed through the Node.js test runner (`npx hardhat test`).

4.5.1 Test Architecture and Isolation Strategy

The test layer choice was driven by a fundamental constraint: the protocol's core submission flow requires *off-chain* ECDSA signature generation to simulate the Issuer backend, which cannot be performed natively inside the EVM. This makes Solidity-level unit tests (`.t.sol`) insufficient for end-to-end validation, necessitating TypeScript tests that integrate off-chain cryptographic logic with on-chain EVM simulation.

To prevent shared-state pollution between test groups, particularly those that manipulate the blockchain clock or modify review flags, each top-level `describe` block instantiates a completely isolated EVM via `network.create()`. Within each isolated network, the full six-contract ecosystem is deployed from scratch, ensuring that every test group starts from a pristine state.

4.5.2 Test Coverage Summary

The suite comprises exactly **26 rigorous integration tests** that exercise not only the standard functional flows (happy paths) but, critically, the malicious attack vectors defined in the threat model. All 26 tests pass deterministically.

Category	Test Focus	Verified Behaviors & Malicious Cases
Review Submission (8 Tests)	Happy path	Review stored correctly; auto-registration mints 1 RWT.
	Bounds validation	<code>InvalidScore</code> reverts for scores ≤ 0 or ≥ 6 .
	Fraud protection	<code>UntrustedIssuer</code> reverts if the Product ID is tampered with (mismatches the signature). <code>NotReviewer</code> blocks Identity Theft (using another user's DID).
	Replay protection	Double Spend prevented: <code>AlreadySpent</code> revert on Nullifier reuse.
Modification & Revocation (7 Tests)	Authorization	<code>NotReviewer</code> reverts if a third party attempts to modify or revoke someone else's review.
	State limits	Modification window expiry enforced. <code>ModificationLimitReached</code> prevents spamming > 1 edits. <code>RevocationWindowElapsed</code> blocks delayed revocations.
	Window reset	A modification resets <code>windowStart</code> to the modification timestamp, opening a fresh 30-day revocation window. Verified that revocation succeeds when invoked within 30 days of the modification even if more than 30 days have elapsed since the original submission.
Vendor Reply (2 Tests)	Reply Verification	Vendor reply logged correctly. <code>ReviewAlreadyRevoked</code> prevents replying to deleted reviews.
Curation and Rewards (4 Tests)	Voting integrity	Upvotes cast with valid PoP. <code>UntrustedIssuer</code> blocks voters with invalid signatures.
	Time-lock bounds	<code>CurationWindowStillOpen</code> strictly enforced before 30 days.
	Reward distribution	Successful finalization after 31-day time travel correctly updates reputations and mints tokens.
Economic Layer (2 Tests)	Burn-to-Redeem	<code>BalanceBelowThreshold</code> prevents redemption below T_{min} .
	Soulbound SBT	<code>transfer()</code> , <code>transferFrom()</code> , <code>approve()</code> all strictly revert to prevent reputation trading.
DID Liveness (1 Test)	Registry Checks	<code>NotRegistered</code> halts transactions for inactive DIDs.
DID Issuer Registration (2 Tests)	Happy path	A DID registered via <code>issuerRegisterDID</code> resolves correctly to the target owner address (not the issuer).
	Access control	<code>NotIssuer</code> reverts when a non-authorized account invokes <code>issuerRegisterDID</code> .

Table 4.2: Integration test matrix (26 test cases, all passing), highlighting the security-focused assertions against malicious inputs.

The concise and highly targeted nature of the test suite is a direct result of the strictly

decentralized Direct Submit architecture. By leveraging EVM-native ECDSA verification without intermediate entities, the attack surface is minimized, allowing the test suite to focus entirely on core protocol mechanics and direct user interactions. The resulting 26 tests represent a 100% functional coverage of the modernized, permissionless architecture.

4.5.3 Off-Chain Signature Generation in Tests

Each test that involves a review submission or a vote must first generate a valid Proof of Purchase off-chain. The test helper function `buildPoPSig` replicates the exact encoding pipeline of the Issuer backend (Section 4.3): it serializes the payload with `encodeAbiParameters`, computes the `keccak256` hash, and signs it with the issuer's private key using a raw ECDSA signature:

```
async function buildPoPSig(did, vendorDid, productIdHash, nullifier,
  ↪ sdDigests) {
  const encoded = encodeAbiParameters(
    parseAbiParameters("string, string, bytes32, bytes32,
      ↪ bytes32[]"),
    [did, vendorDid, productIdHash, nullifier, sdDigests],
  );
  const hash = keccak256(encoded);
  // Raw sign: no EIP-191 prefix
  const sig = await issuerAccount.sign({ hash });
  return {
    v: parseInt(sig.slice(130, 132), 16),
    r: `0x${sig.slice(2, 66)}` as `0x${string}`,
    s: `0x${sig.slice(66, 130)}` as `0x${string}`,
  };
}
```

4.5.4 Curation Window Simulation (Time Travel)

The 30-day curation window presents an obvious obstacle to rapid testing. To overcome it, the test suite leverages Hardhat's `networkHelpers.time.increase()` API, which instructs the local EVM node to advance the blockchain's internal clock by a specified number of seconds. In the curation test group, the clock is advanced by 31 days (2,678,400 seconds) immediately after the voting phase:

```
// Advance blockchain clock by 31 days
await networkHelpers.time.increase(31 * 24 * 60 * 60);

// Now finalizeCuration succeeds
await viem.assertions.emit(
  reviewContract.write.finalizeCuration([reviewId]),
  reviewContract, "CurationFinalized"
```

);

This technique enables the validation of all time-dependent mechanics (the curation window guard, the modification window expiry, and the reward distribution) without modifying the underlying smart contract logic or waiting in real time.

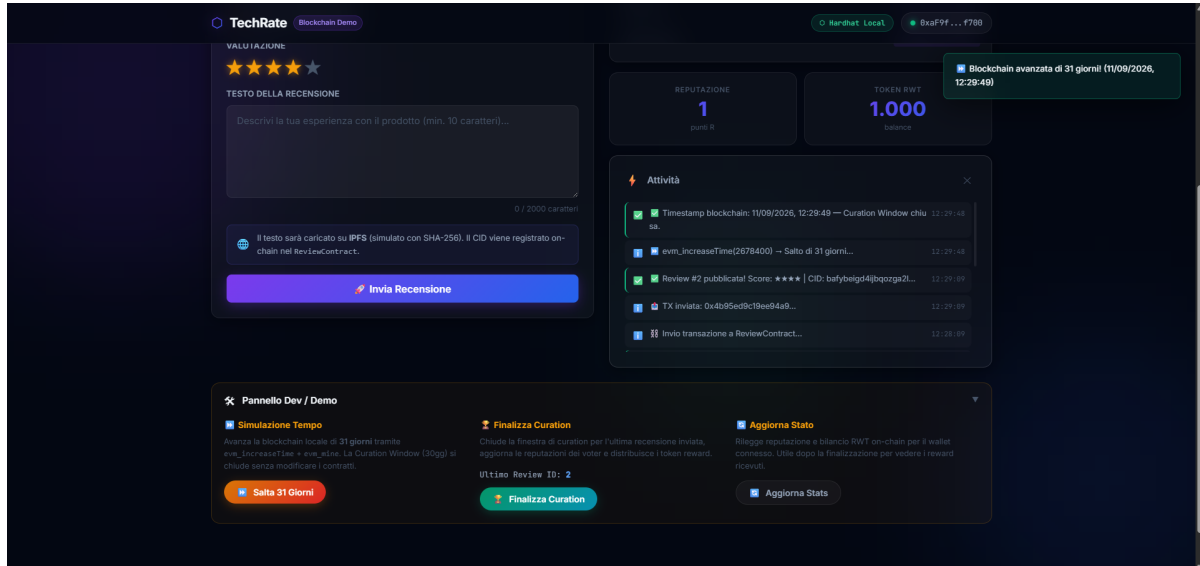


Figure 4.4: Curation window simulation - the time travel (31 days next)

4.5.5 Test Execution Results

The final test run produces:

Running node:test tests

S2.3 - submitReview

[OK] happy path: user submits review directly, emits

↪ ReviewSubmitted

[OK] auto-registers reviewer and mints 1 RWT welcome token (R0 =

↪ 1)

[OK] reverts InvalidScore for score = 0

[OK] reverts InvalidScore for score > 5

[OK] reverts UntrustedIssuer if productId mismatches the one

↪ signed

[OK] reverts NotReviewer if user tries to submit using another DID

[OK] reverts on nullifier replay - double-spend protection

[OK] reverts UntrustedIssuer when signature is invalid

DID - liveness enforcement

[OK] reverts NotRegistered if reviewer has no registered DID

DIDRegistry - issuerRegisterDID

- [OK] happy path: DID registered by issuer resolves to the correct
↪ owner address
- [OK] reverts NotIssuer when called by a non-issuer account

S2.4 - modifyReview

- [OK] allows a single modification within the 3-hour window
- [OK] reverts NotReviewer if someone else tries to modify
- [OK] reverts ModificationLimitReached if trying to modify twice

S2.4 - revokeReview

- [OK] allows revocation and sets the revoked flag
- [OK] reverts NotReviewer if someone else tries to revoke
- [OK] reverts RevocationWindowElapsed if trying to revoke after 30
↪ days
- [OK] modification resets windowStart: revocation allowed within
↪ 30 days of modification

S2.4 - submitVendorReply

- [OK] vendor can submit a reply and emits VendorReplySubmitted
- [OK] reverts ReviewAlreadyRevoked if the review was revoked

S2.6 - voteOnReview / finalizeCuration

- [OK] user casts an upvote directly
- [OK] reverts UntrustedIssuer when non-issuer tries to vote
- [OK] reverts CurationWindowStillOpen if finalization is called
↪ early
- [OK] finalizes curation after 31-day time travel

S2.6 - redeemTokens

- [OK] reverts BalanceBelowThreshold for a user below T_min

ReputationToken - Soulbound

- [OK] reverts on transfer()

26 passing (26 nodejs)

All 26 test cases pass, confirming that every execution path specified in WP2 (from the initial review submission through curation, reward distribution, reputation aggregation, and token redemption) has been correctly implemented and is enforced by the on-chain logic.

List of Figures

2.1	Review lifecycle and state transitions.	18
4.1	Initial window	46
4.2	Wallet connection	47
4.3	Review and Submitting	48
4.4	Curation window simulation - the time travel (31 days next)	52

List of Tables

3.1	Pseudonymity guarantees by observer class.	28
3.2	Summary of property-by-property verification and adversary barriers. . .	29
3.3	Summary of identified threat vectors, architectural mitigations, and accepted residual risks.	36
4.1	Summary of the technology stack utilized for the Proof of Concept implementation.	38
4.2	Integration test matrix (26 test cases, all passing), highlighting the security-focused assertions against malicious inputs.	50